

TALLER III CORTE II

SEMANA 11

DOCENTE

ELIECER MONTERO OJEDA

INTEGRANTE

JUAN DAVID RENGIFO CHAVEZ

UNIDADES TECNOLOGICAS DE SANTANDER
PATRONES DE SOFTWARE
INGENIERIA EN SISTEMAS
2026

Tabla de contenido

1. INTRODUCCIÓN	3
2. OBJETIVO GENERAL.....	3
3. DESCRIPCIÓN GENERAL DEL SISTEMA	3
4. DIAGRAMA DE CLASES – MODELO DE DOMINIO	4
5. ARQUITECTURA DEL SISTEMA	4
6. PATRONES CREACIONALES	6
6.1 PATRÓN SINGLETON (PREDICTIONENGINE)	6
6.2. PATRÓN FACTORY METHOD	9
6.3 PATRÓN ABSTRACT FACTORY	16
6.4. PATRÓN FLUENT BUILDER (TRANSACTION).....	28
6.5. PATRÓN PROTOTYPE (ENERGYOFFER)	40
7. PATRONES ESTRUCTURALES	49
7.1. PATRÓN ADAPTER (IOT)	49
7.2. PATRÓN BRIDGE (IOT)	61
7.3. PATRÓN DECORATOR (TRANSACTION).....	70
7.4. PATRÓN FACADE (ENERGIA)	80
7.5. PATRÓN COMPOSITE (IOT).....	89
10. REFERENCIAS BIBLIOGRÁFICAS	98

1. INTRODUCCIÓN

El crecimiento de la generación distribuida de energía, especialmente a través de fuentes renovables como paneles solares residenciales, ha transformado el rol tradicional de los usuarios dentro del sistema eléctrico. Actualmente, los hogares no solo consumen energía, sino que también pueden producir excedentes que podrían ser aprovechados por otros usuarios.

En este contexto surge Enerlink, una plataforma de intercambio energético diseñada para permitir la compra y venta de excedentes de energía entre usuarios, incorporando mecanismos de subasta, integración con dispositivos IoT domésticos y funcionalidades de predicción de producción y consumo energético.

El desarrollo de Enerlink tiene un enfoque estrictamente académico y se orienta a la aplicación práctica de patrones de diseño GoF (Gang of Four), arquitectura hexagonal, pruebas automatizadas y buenas prácticas de ingeniería de software. El proyecto busca demostrar cómo un sistema de comercio energético puede estructurarse de manera modular, extensible y mantenible mediante el uso de principios de diseño orientados a objetos.

2. OBJETIVO GENERAL

Implementar la plataforma Enerlink aplicando arquitectura hexagonal y patrones de diseño GoF, garantizando una solución modular, extensible y validada mediante pruebas automatizadas.

2.1 Objetivos Específicos

- Diseñar la arquitectura y el modelo de dominio de Enerlink mediante diagramas UML y la definición formal de patrones de diseño, asegurando la correcta separación de responsabilidades bajo el enfoque de arquitectura hexagonal.
- Implementar la plataforma Enerlink en Java utilizando Spring Boot, integrando progresivamente los patrones de diseño GoF definidos y garantizando la coherencia con la arquitectura planteada.
- Evaluar la calidad y el comportamiento del sistema mediante pruebas automatizadas, asegurando una cobertura mínima del 80% y verificando el cumplimiento de los requisitos funcionales y estructurales.

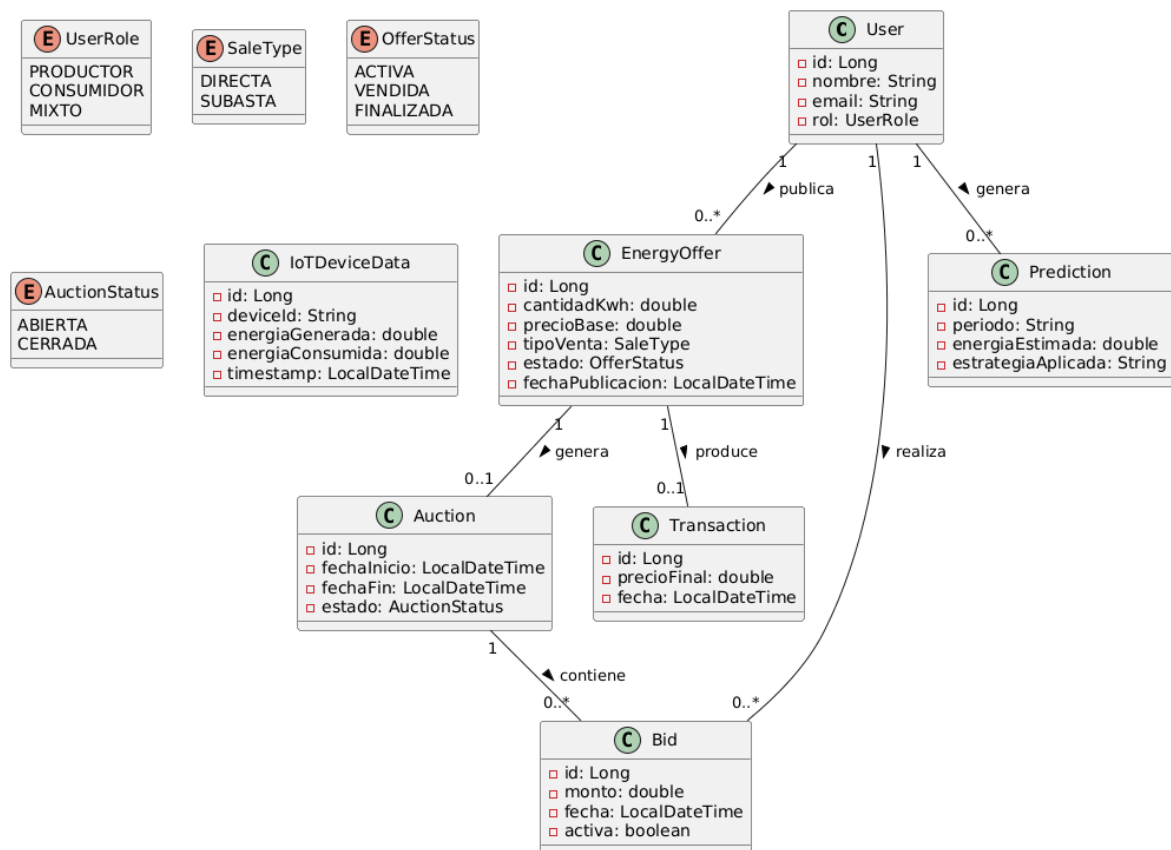
3. DESCRIPCIÓN GENERAL DEL SISTEMA

Enerlink es una plataforma backend orientada al intercambio de energía entre usuarios productores y consumidores. El sistema permite a los productores publicar excedentes energéticos bajo modalidad de venta directa o subasta, mientras que los consumidores pueden adquirir energía según las condiciones establecidas.

El sistema contempla los siguientes módulos funcionales:

1. **Gestión de Usuarios:** registro y administración de usuarios con roles de productor, consumidor o mixto.
2. **Publicación de Ofertas Energéticas:** registro de excedentes energéticos disponibles para venta.
3. **Sistema de Subastas:** gestión de pujas y determinación de ganadores.
4. **Compra Directa:** adquisición inmediata de energía al precio base establecido.
5. **Integración IoT:** recepción y adaptación de datos energéticos provenientes de dispositivos domésticos.
6. **Predicción Energética:** estimación de producción o consumo mediante estrategias configurables.

4. DIAGRAMA DE CLASES – MODELO DE DOMINIO



5. ARQUITECTURA DEL SISTEMA

5.1 Estilo Arquitectónico Adoptado

La plataforma **Enerlink** se desarrolla bajo el enfoque de **Arquitectura Hexagonal**, también conocida como arquitectura de Puertos y Adaptadores. Este estilo

arquitectónico permite separar claramente la lógica de negocio del sistema de los detalles de infraestructura, como bases de datos, controladores web o integraciones externas.

El principio fundamental de la arquitectura hexagonal consiste en ubicar el dominio en el núcleo del sistema, aislándolo de dependencias externas mediante la definición de puertos (interfaces) y adaptadores (implementaciones concretas).

Este enfoque favorece:

- Alta cohesión en la lógica de negocio.
- Bajo acoplamiento con tecnologías específicas.
- Facilidad de pruebas unitarias.
- Extensibilidad y mantenibilidad del sistema.

5.2 Estructura Arquitectónica de Enerlink

Enerlink se organiza en tres capas principales:

5.2.1 Dominio

Contiene:

- Entidades del negocio.
- Reglas de negocio.
- Enumeraciones.
- Lógica central del sistema.

Esta capa no depende de ningún framework ni tecnología externa.

5.2.2 Aplicación

Contiene:

- Casos de uso.
- Coordinación de operaciones del sistema.
- Orquestación de entidades del dominio.

Esta capa define los **puertos** que permiten interactuar con el dominio desde el exterior.

5.2.3 Infraestructura

Contiene:

- Controladores REST.
- Implementaciones de persistencia.
- Adaptadores IoT.

- Configuración del framework.
- Logging y monitoreo.

Esta capa implementa los adaptadores que permiten la comunicación con el entorno externo.

6. PATRONES CREACIONALES

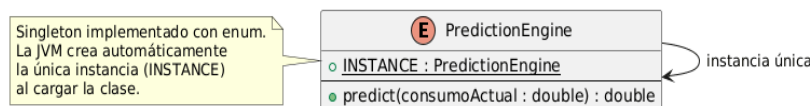
6.1 PATRÓN SINGLETON (PREDICTIONENGINE)

6.1.1 Descripción del patrón

El patrón Singleton garantiza que exista una única instancia de una clase durante el ciclo de vida de la aplicación y que dicha instancia sea accesible de manera controlada.

En este proyecto, el patrón se implementó mediante un enum, siguiendo la recomendación de Joshua Bloch en *Effective Java*.

6.1.2 Diseño UML



6.1.3 Implementación

```

1 package com.enerlink.enerlink.configuracion;
2
3 import java.util.concurrent.ThreadLocalRandom;
4
5 /**
6  * PATRÓN SINGLETON - Implementación usando Enum (Joshua Bloch)
7  *
8  * Esta es la forma recomendada en Effective Java.
9  *
10 * Ventajas:
11 * - Thread-safe automáticamente
12 * - Protección contra serialización
13 * - Protección contra reflexión
14 * - Implementación simple y robusta
15 *
16 * No depende de Spring.
17 */
18
19 public enum PredictionEngine {
20
21     INSTANCE; // Única instancia garantizada por la JVM
22
23     /**
24      * Método de negocio: predice el consumo futuro.
25      *
26      * @param consumoActual consumo actual en kWh
27      * @return predicción de consumo futuro
28      */
29     public double predict(double consumoActual) {
30
31         double factorBase = 1.15;
32
33         // Variación aleatoria entre -5% y +5%
34         double variacion = ThreadLocalRandom.current().nextDouble(-0.05, 0.05);
35
36         double factorFinal = factorBase + variacion;
37
38         return consumoActual * factorFinal;
39     }
40 }
  
```

6.1.4 Justificación de la implementación

La clase PredictionEngine implementa el patrón **Singleton** mediante un enum, siguiendo la recomendación de Joshua Bloch en *Effective Java*. Esta implementación garantiza que la **JVM cree una única instancia** de PredictionEngine, evitando la posibilidad de generar instancias adicionales mediante reflexión o serialización.

El acceso a la instancia se realiza de manera simple:

PredictionEngine.INSTANCE

No existe constructor público ni método getInstance(), ya que el propio mecanismo del enum controla la instanciación.

Desde el punto de vista técnico:

- La clase no mantiene estado mutable; el método predict utiliza únicamente variables locales, lo que asegura **seguridad en entornos multihilo**.
- Cumple con el principio de **Responsabilidad Única**, ya que su única función es calcular predicciones de consumo.
- No introduce **acoplamientos innecesarios**, ya que no depende de Spring ni de otras capas del proyecto.

Desde el punto de vista de negocio en Enerlink:

- PredictionEngine es un **servicio compartido sin estado**, utilizado por múltiples componentes del sistema, como UserService.
- Tener una única instancia asegura que las predicciones sean consistentes en todo el proyecto, evitando discrepancias entre diferentes cálculos.
- Reduce el consumo de memoria y simplifica la lógica, al eliminar la necesidad de crear, inyectar o gestionar múltiples instancias.
- Facilita la mantenibilidad y la extensión del proyecto, ya que cualquier cambio en la lógica de predicción se realiza en un solo lugar.

En resumen, la implementación **cumple los objetivos del patrón Singleton**, garantiza consistencia, eficiencia y simplicidad, y aporta **beneficios claros tanto desde el diseño técnico como desde la operación del sistema**.

6.1.5 Prueba de funcionamiento

```
1 package com.enerLink.enerLink.configuracion;
2
3 import org.springframework.boot.CommandLineRunner;
4 import org.springframework.stereotype.Component;
5
6 @Component
7 public class SingletonDemoRunner implements CommandLineRunner {
8
9     @Override
10    public void run(String... args) throws Exception {
11        PredictionEngine engine1 = PredictionEngine.INSTANCE;
12        PredictionEngine engine2 = PredictionEngine.INSTANCE;
13        PredictionEngine engine3 = PredictionEngine.INSTANCE;
14        PredictionEngine engine4 = PredictionEngine.INSTANCE;
15
16        System.out.println("=====");
17        System.out.println("        DEMO PATRÓN SINGLETON        ");
18        System.out.println("=====\\n");
19
20        System.out.println("Verificación de que todas las instancias son la misma:");
21        System.out.println("-----");
22        System.out.printf("engine1 == engine2 ? %b\\n", engine1 == engine2);
23        System.out.printf("engine1 == engine3 ? %b\\n", engine1 == engine3);
24        System.out.printf("engine1 == engine4 ? %b\\n", engine1 == engine4);
25
26        System.out.println("\\nUso del método predict():");
27        System.out.println("-----");
28        double consumoActual = 100.0;
29        double prediccion = engine1.predict(consumoActual);
30        System.out.printf("Consumo actual: %.2f kWh\\n", consumoActual);
31        System.out.printf("Predicción de consumo: %.2f kWh\\n", prediccion);
32
33        System.out.println("\\n=====");
34        System.out.println("                FIN DE LA DEMO SINGLETON                ");
35        System.out.println("=====\\n");
36    }
```

```
=====
        DEMO PATRÓN SINGLETON
=====

Verificación de que todas las instancias son la misma:
-----
engine1 == engine2 ? true
engine1 == engine3 ? true
engine1 == engine4 ? true

Uso del método predict():
-----
Consumo actual: 100,00 kWh
Predicción de consumo: 111,79 kWh

=====
                FIN DE LA DEMO SINGLETON
=====
```

Como se puede observar, todas las variables creadas (engine 1, 2, 3...) apuntan a una misma instancia y permite utilizar su método predict(). Lo que significa que el patrón singleton está funcionando correctamente dentro del proyecto.

6.1.6 Aclaración sobre la implementación actual

Por motivos de **practicidad y enfoque académico**, la clase PredictionEngine no implementa un algoritmo real de predicción de consumo. El objetivo principal de la implementación actual es **demostrar correctamente el patrón Singleton**, garantizando:

- Instancia única compartida.
- Seguridad en entornos multihilo.
- Acceso uniforme desde cualquier parte del sistema.

A futuro, se planea reemplazar el método predict por un **algoritmo real de predicción**, capaz de analizar más variables y atributos relevantes para el consumo energético. En ese momento, el PredictionEngine podrá evolucionar para:

- Manejar más parámetros de entrada y modelos complejos.
- Mantener la instancia única, asegurando consistencia en todas las llamadas.
- Adaptarse a necesidades de cálculo más avanzadas sin romper la arquitectura ni el patrón Singleton.

La versión actual es **prototipo funcional del patrón**, mientras que la lógica de negocio del cálculo de predicciones será ampliada en versiones futuras del sistema.

6.2. PATRÓN FACTORY METHOD

6.2.1 Descripción del patrón

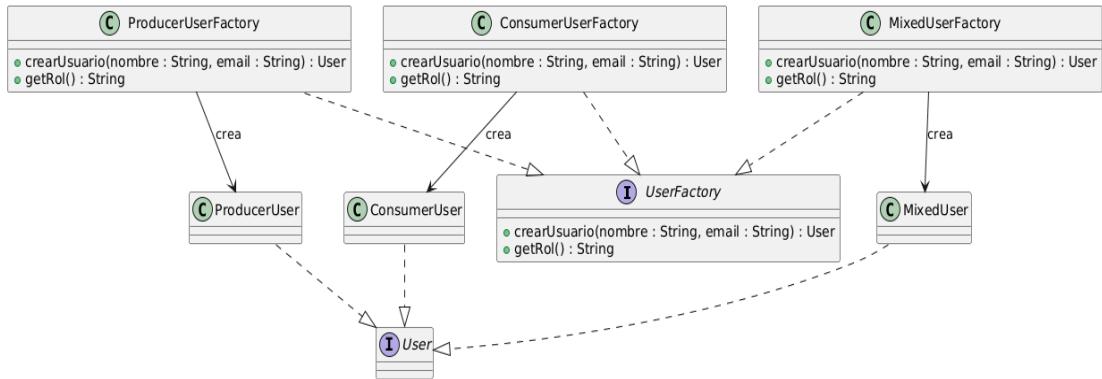
El **Factory Method** es un patrón de diseño creacional que define un contrato para crear objetos, pero permite que las subclases decidan **qué clase concreta instanciar**.

En Enerlink, este patrón se utiliza para que el UserService pueda **crear distintos tipos de usuarios** (ProducerUser, ConsumerUser, MixedUser) sin depender directamente de sus clases concretas. Esto asegura que la capa de aplicación permanezca **desacoplada de los detalles de implementación del dominio**, facilitando mantenimiento y extensibilidad.

Desde el punto de vista de diseño, el patrón favorece:

- **Open/Closed Principle**: nuevos tipos de usuario se pueden agregar sin modificar UserService.
- **Single Responsibility Principle**: cada factory se encarga únicamente de crear un tipo de usuario.
- **Dependency Inversion Principle**: el servicio depende de la abstracción UserFactory y no de implementaciones concretas.

6.2.2 Diagrama UML



6.2.3 Implementación

Clase abstracta User

```
1 package com.enerlink.enerlink.usuario.dominio.modelo;
2
3 /**
4  * Clase abstracta del dominio. No depende de Spring.
5  * Define el contrato común de cualquier tipo de usuario en Enerlink.
6  */
7 public abstract class User {
8
9     protected Long id;
10    protected String nombre;
11    protected String email;
12    protected String rol; // "PRODUCER", "CONSUMER" o "MIXED"
13
14    public User(String nombre, String email, String rol) {
15        this.nombre = nombre;
16        this.email = email;
17        this.rol = rol;
18    }
19
20    public String describir() {
21        return "Usuario: " + nombre + " con rol: " + rol;
22    }
23
24    // Getters
25    public Long getId() { return id; }
26    public String getNombre() { return nombre; }
27    public String getEmail() { return email; }
28    public String getRol() { return rol; }
29
30    public void setId(Long id) { this.id = id; }
31 }
```

Clase concreta ProducerUser

```
1 package com.enerlink.enerlink.usuario.dominio.modelo;
2
3 /**
4  * Usuario productor de energía.
5  * Producto concreto del patrón Factory Method.
6  */
7 public class ProducerUser extends User {
8
9     public ProducerUser(String nombre, String email) {
10         super(nombre, email, "PRODUCER");
11     }
12
13     @Override
14     public String describir() {
15         return "Soy " + nombre + " y produzco energía en Enerlink.";
16     }
17 }
```

Clase concreta ConsumerUser

```
1 package com.enerlink.enerlink.usuario.dominio.modelo;
2
3 /**
4  * Usuario productor de energía.
5  * Producto concreto del patrón Factory Method.
6  */
7 public class ProducerUser extends User {
8
9     public ProducerUser(String nombre, String email) {
10         super(nombre, email, "PRODUCER");
11     }
12
13     @Override
14     public String describir() {
15         return "Soy " + nombre + " y produzco energía en Enerlink.";
16     }
17 }
```

Clase concreta MixedUser

```
1 package com.enerlink.enerlink.usuario.dominio.modelo;
2
3 /**
4  * Usuario consumidor de energía.
5  * Producto concreto del patrón Factory Method.
6  */
7 public class MixedUser extends User {
8
9     public MixedUser(String nombre, String email) {
10         super(nombre, email, "MIXED");
11     }
12
13     @Override
14     public String describir() {
15         return "Soy " + nombre + ", Produzco y consumo energía en Enerlink.";
16     }
17 }
```

Clase abstracta UserFactory

```
1 package com.enerlink.enerlink.usuario.dominio.factory;
2
3 import com.enerlink.enerlink.usuario.dominio.modelo.User;
4
5 /**
6  * FACTORY METHOD CLÁSICO
7  *
8  * Clase abstracta que define el contrato de cualquier Factory de usuarios.
9  * Cada subclase concreta decide qué tipo de usuario crear.
10 */
11 public abstract class UserFactory {
12
13     /**
14      * Método factory abstracto: cada subclase concreta implementa la creación de su usuario.
15      *
16      * @param nombre nombre del usuario
17      * @param email correo del usuario
18      * @return instancia concreta de User
19      */
20     public abstract User crearUsuario(String nombre, String email);
21
22 }
```

Clase concreta ProducerUserFactory

```
1 package com.enerlink.enerlink.usuario.dominio.factory;
2
3 import org.springframework.stereotype.Component;
4
5 import com.enerlink.enerlink.usuario.dominio.modelo.ProducerUser;
6 import com.enerlink.enerlink.usuario.dominio.modelo.User;
7
8 /**
9  * Factory concreto que crea usuarios productores de energía.
10  * Producto concreto del patrón Factory Method.
11  */
12 @Component("PRODUCER")
13 public class ProducerUserFactory extends UserFactory {
14
15     @Override
16     public User crearUsuario(String nombre, String email) {
17         return new ProducerUser(nombre, email);
18     }
19 }
```

Clase concreta ConsumerUserFactory

```
1 package com.enerlink.enerlink.usuario.dominio.factory;
2
3 import org.springframework.stereotype.Component;
4
5 import com.enerlink.enerlink.usuario.dominio.modelo.ConsumerUser;
6 import com.enerlink.enerlink.usuario.dominio.modelo.User;
7
8 /**
9  * Factory concreto que crea usuarios consumidores de energía.
10  * Producto concreto del patrón Factory Method.
11  */
12 @Component("CONSUMER")
13 public class ConsumerUserFactory extends UserFactory {
14
15     @Override
16     public User crearUsuario(String nombre, String email) {
17         return new ConsumerUser(nombre, email);
18     }
19 }
```

Clase concreta MixedUserFactory

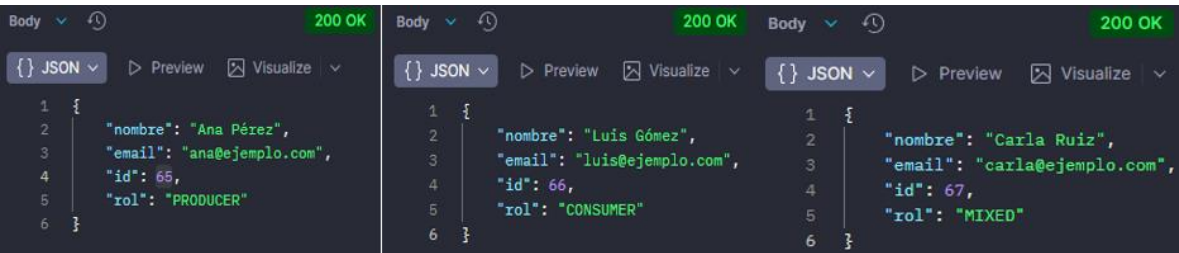
```
1 package com.enerlink.enerlink.usuario.dominio.factory;
2
3 import org.springframework.stereotype.Component;
4
5 import com.enerlink.enerlink.usuario.dominio.modelo.MixedUser;
6 import com.enerlink.enerlink.usuario.dominio.modelo.User;
7
8 /**
9  * Factory concreto que crea usuarios mixtos (producen y consumen energía).
10  * Producto concreto del patrón Factory Method.
11  */
12 @Component("MIXED")
13 public class MixedUserFactory extends UserFactory {
14
15     @Override
16     public User crearUsuario(String nombre, String email) {
17         return new MixedUser(nombre, email);
18     }
19 }
```

6.2.4 Prueba de funcionamiento

Creación de usuarios por rol:

Se usó el endpoint **POST /api/usuarios** para crear un usuario de cada tipo: **Producer**, **Consumer** y **Mixed**:

Rol	JSON enviado
Producer	{ "nombre": "Ana Pérez", "email": "ana@ejemplo.com", "rol": "PRODUCER" }
Consumer	{ "nombre": "Luis Gómez", "email": "luis@ejemplo.com", "rol": "CONSUMER" }
Mixed	{ "nombre": "Carla Ruiz", "email": "carla@ejemplo.com", "rol": "MIXED" }



Listar todos los usuarios:

Para comprobar que los usuarios se crearon correctamente y que cada instancia mantiene su comportamiento polimórfico, se llamó al endpoint **GET /api/usuarios**.



Como se puede observar, se logró crear y listar usuarios con distintos roles, demostrando que el patrón Factory maneja correctamente la creación de los distintos tipos de usuarios que existen.

6.2.5 Justificación de la implementación

Técnicamente:

- UserService nunca hace new ProducerUser(...) ni new ConsumerUser(...).
- La creación de usuarios se delega a la factory correspondiente según el rol, garantizando que el servicio no dependa de clases concretas.
- Si se requiere un nuevo tipo de usuario, solo hay que agregar una nueva factory y registrarla en el mapa de factories, sin modificar UserService.

Desde la perspectiva del negocio en Enerlink:

- Cada usuario puede tener un comportamiento propio (por ejemplo, describir()).
- La fábrica asegura que el UserService reciba siempre el tipo correcto, garantizando consistencia y mantenimiento fácil.
- Reduce errores al centralizar la lógica de creación y mejora la capacidad de escalar el sistema.

Relación con Clean Architecture y SOLID:

- La capa de aplicación depende de **abstracciones** (UserFactory) y no de clases concretas.
- Cada factory tiene **una única responsabilidad**: crear un tipo de usuario concreto.
- El sistema es **abierto a extensiones y cerrado a modificaciones**, cumpliendo Open/Closed Principle.

6.2.6 Aclaración sobre la implementación actual

Actualmente, las factories devuelven usuarios con datos básicos (nombre, email, rol) y su método describir(). Esto es suficiente para demostrar el patrón Factory Method sin introducir lógica de negocio compleja en la creación de usuarios.

A futuro, si se requiere agregar configuraciones iniciales adicionales o atributos dinámicos en los usuarios, las factories podrán evolucionar para incluirlos sin afectar UserService ni romper el patrón.

6.3 PATRÓN ABSTRACT FACTORY

6.3.1 Descripción del patrón

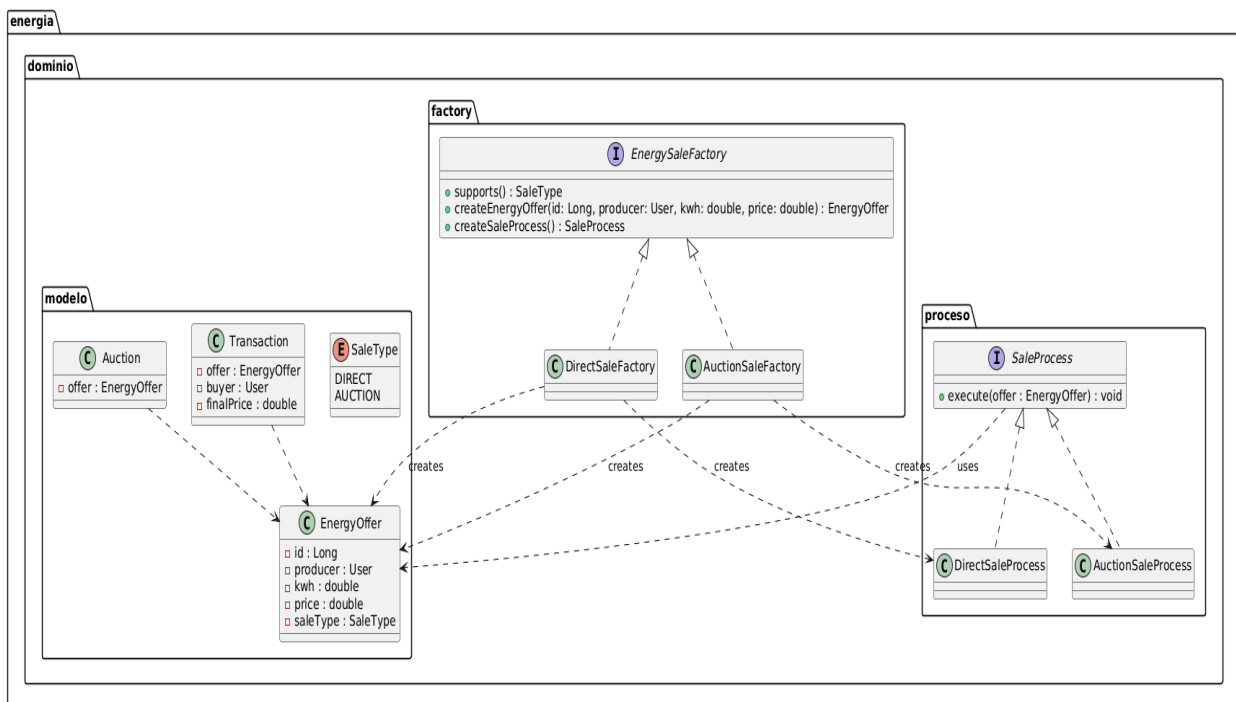
El Abstract Factory es un patrón de diseño creacional que define un contrato para crear familias de objetos relacionados, garantizando que las combinaciones de productos sean coherentes entre sí sin acoplarse a clases concretas.

En Enerlink, este patrón se utiliza para que la capa de aplicación pueda crear ofertas de energía y procesos de venta consistentes (venta directa o subasta) sin depender directamente de DirectSaleProcess, AuctionSaleProcess ni de constructores concretos de EnergyOffer.

Desde el punto de vista de diseño, el patrón favorece:

- Open/Closed Principle: nuevos tipos de venta (por ejemplo, venta programada) se pueden agregar creando nuevas fábricas concretas sin modificar el código existente del servicio que orquesta las ventas.
- Single Responsibility Principle: cada fábrica concreta (DirectSaleFactory, AuctionSaleFactory) se encarga únicamente de crear la familia de objetos correspondiente a su tipo de venta.
- Dependency Inversion Principle: la capa de aplicación depende de la abstracción EnergySaleFactory y del contrato SaleProcess, en lugar de acoplarse a clases concretas, lo que facilita pruebas, mantenimiento y extensibilidad.

6.3.2 Diagrama UML



6.3.3 Implementación

Clase EnergyOffer

```
1 package com.enerlink.enerlink.energia.dominio.modelo;
2
3 import com.enerlink.enerlink.usuario.dominio.modelo.User;
4
5 public class EnergyOffer {
6
7     private Long id;
8     private User producer;
9     private double kwh;
10    private double price;
11    private SaleType saleType;
12
13    public EnergyOffer(Long id, User producer, double kwh, double price, SaleType saleType) {
14        this.id = id;
15        this.producer = producer;
16        this.kwh = kwh;
17        this.price = price;
18        this.saleType = saleType;
19    }
20 }
```

EnergyOffer representa una oferta energética publicada por un productor, indicando cantidad en kWh, precio y tipo de venta (DIRECT o AUCTION).

Clase enum SaleType

```
1 package com.enerlink.enerlink.energia.dominio.modelo;
2
3 public enum SaleType {
4     DIRECT,
5     AUCTION
6 }
```

SaleType define las variantes soportadas por el sistema: venta directa y subasta.

Clase Auction

```
1 package com.enerlink.enerlink.energia.dominio.modelo;
2
3 public class Auction {
4
5     private EnergyOffer offer;
6
7     public Auction(EnergyOffer offer) {
8         this.offer = offer;
9     }
10 }
11 }
```

Auction modela la subasta asociada a una oferta configurada como subasta.

Clase Transaction

```

1 package com.enerlink.enerlink.energia.dominio.modelo;
2
3 import com.enerlink.enerlink.usuario.dominio.modelo.User;
4
5 public class Transaction {
6
7     private EnergyOffer offer;
8     private User buyer;
9     private double finalPrice;
10
11     public Transaction(EnergyOffer offer, User buyer, double finalPrice) {
12         this.offer = offer;
13         this.buyer = buyer;
14         this.finalPrice = finalPrice;
15     }
16 }
17

```

Transaction representa la compra final de una oferta, ya sea por venta directa o por subasta.

Interfaz de la fábrica abstracta

```

1 package com.enerlink.enerlink.energia.dominio.factory;
2
3 import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;
4 import com.enerlink.enerlink.energia.dominio.modelo.SaleType;
5 import com.enerlink.enerlink.energia.dominio.proceso.SaleProcess;
6 import com.enerlink.enerlink.usuario.dominio.modelo.User;
7
8 public interface EnergySaleFactory {
9
10     SaleType supports();
11
12     EnergyOffer createEnergyOffer(Long id, User producer, double kwh, double price);
13
14     SaleProcess createSaleProcess();
15
16 }
17

```

- supports() indica para qué tipo de venta es responsable la fábrica concreta.
- createEnergyOffer(...) crea una oferta coherente con el tipo de venta de la fábrica.
- createSaleProcess() devuelve el proceso de venta apropiado (DirectSaleProcess o AuctionSaleProcess).

Fabrica concreta DirectSaleFactory

```
1 package com.enerlink.enerlink.energia.dominio.factory;
2
3 import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;
4 import com.enerlink.enerlink.energia.dominio.modelo.SaleType;
5 import com.enerlink.enerlink.energia.dominio.proceso.DirectSaleProcess;
6 import com.enerlink.enerlink.energia.dominio.proceso.SaleProcess;
7 import com.enerlink.enerlink.usuario.dominio.modelo.User;
8
9 public class DirectSaleFactory implements EnergySaleFactory {
10
11     @Override
12     public SaleType supports() {
13         return SaleType.DIRECT;
14     }
15
16     @Override
17     public EnergyOffer createEnergyOffer(Long id, User producer, double kwh, double price) {
18         return new EnergyOffer(id, producer, kwh, price, SaleType.DIRECT);
19     }
20
21     @Override
22     public SaleProcess createSaleProcess() {
23         return new DirectSaleProcess();
24     }
25 }
26
```

DirectSaleFactory produce una EnergyOffer configurada como venta directa y un DirectSaleProcess para ejecutar la compra inmediata.

Fabrica concreta AuctionSaleFactory

```
1 package com.enerlink.enerlink.energia.dominio.factory;
2
3 import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;
4 import com.enerlink.enerlink.energia.dominio.modelo.SaleType;
5 import com.enerlink.enerlink.energia.dominio.proceso.AuctionSaleProcess;
6 import com.enerlink.enerlink.energia.dominio.proceso.SaleProcess;
7 import com.enerlink.enerlink.usuario.dominio.modelo.User;
8
9 public class AuctionSaleFactory implements EnergySaleFactory {
10
11     @Override
12     public SaleType supports() {
13         return SaleType.AUCTION;
14     }
15
16     @Override
17     public EnergyOffer createEnergyOffer(Long id, User producer, double kwh, double price) {
18         return new EnergyOffer(id, producer, kwh, price, SaleType.AUCTION);
19     }
20
21     @Override
22     public SaleProcess createSaleProcess() {
23         return new AuctionSaleProcess();
24     }
25 }
26
```

AuctionSaleFactory produce una EnergyOffer del tipo subasta y un AuctionSaleProcess que encapsula la lógica de subasta.

Interfaz SalesProcess

```
1 package com.enerlink.enerlink.energia.dominio.proceso;
2
3 import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;
4
5 public interface SaleProcess {
6
7     void execute(EnergyOffer offer);
8
9 }
10
```

Define el contrato común para ejecutar un proceso de venta sobre una oferta de energía.

Implementación DirectSaleProcess

```
1 package com.enerlink.enerlink.energia.dominio.proceso;
2
3 import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;
4
5 public class DirectSaleProcess implements SaleProcess {
6
7     @Override
8     public void execute(EnergyOffer offer) {
9         // lógica de venta directa
10        // aquí se crearía la Transaction
11        System.out.println("Ejecutando venta directa...");
12    }
13 }
14
```

DirectSaleProcess encapsula la lógica de compra directa, donde la oferta se vende inmediatamente generando una transacción.

Implementación AuctionSaleProcess

```
1 package com.enerlink.enerlink.energia.dominio.proceso;
2
3 import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;
4
5 public class AuctionSaleProcess implements SaleProcess {
6
7     @Override
8     public void execute(EnergyOffer offer) {
9         // lógica de subasta
10        // aquí se crearía la Auction
11        System.out.println("Ejecutando subasta...");
12    }
13 }
14
```

AuctionSaleProcess encapsula la lógica de subasta, en la que se gestionan pujas y se crea una Auction asociada a la oferta.

6.3.4 Pruebas de funcionamiento

Endpoints del sistema

El controlador EnergyOfferController expone los siguientes endpoints para la gestión de ofertas de energía.

Método	Endpoint	Descripción
POST	/api/offers	Crear una nueva oferta
GET	/api/offers	Obtener todas las ofertas
GET	/api/offers/{id}	Obtener una oferta por ID
PUT	/api/offers/{id}	Actualizar una oferta
DELETE	/api/offers/{id}	Eliminar una oferta

Casos de prueba

Creación de oferta con venta directa

Esta prueba valida que el sistema utilice la fábrica **DirectSaleFactory** cuando el tipo de venta es **DIRECT**.

Request

- **Método:** POST
- **URL:** http://localhost:8080/api/offers

Body (JSON)

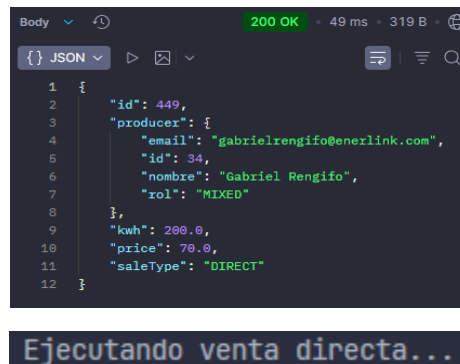
```
{
  "saleType": "DIRECT",
  "producerId": 34,
  "kwh": 200,
  "price": 70
}
```

Resultado esperado

El sistema debe:

1. Seleccionar la fábrica **DirectSaleFactory**.
2. Crear una instancia de **EnergyOffer** con tipo de venta DIRECT.
3. Ejecutar el proceso **DirectSaleProcess**.

Evidencia



Interpretación

Esta prueba demuestra que el patrón Abstract Factory permite crear una familia de objetos coherente para la venta directa sin que el servicio conozca la implementación concreta.

Creación de oferta mediante subasta

Esta prueba válida que el sistema utilice la fábrica **AuctionSaleFactory** cuando el tipo de venta es **AUCTION**.

Request

- **Método:** POST
- **URL:** http://localhost:8080/api/offers

Body

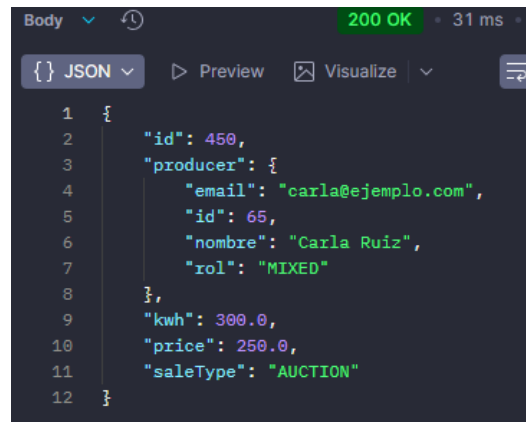
```
{
  "saleType": "AUCTION",
  "producerId": 65,
  "kwh": 300,
  "price": 250
}
```

Resultado esperado

El sistema debe:

1. Seleccionar la fábrica **AuctionSaleFactory**.
2. Crear una oferta configurada como subasta.
3. Ejecutar el proceso **AuctionSaleProcess**.

Evidencia



Ejecutando subasta...

Interpretación

La prueba evidencia que el sistema cambia automáticamente la familia de objetos creados según el tipo de venta, lo cual es el comportamiento esperado del patrón Abstract Factory.

Consulta de todas las ofertas

Esta prueba verifica que el sistema pueda recuperar todas las ofertas registradas.

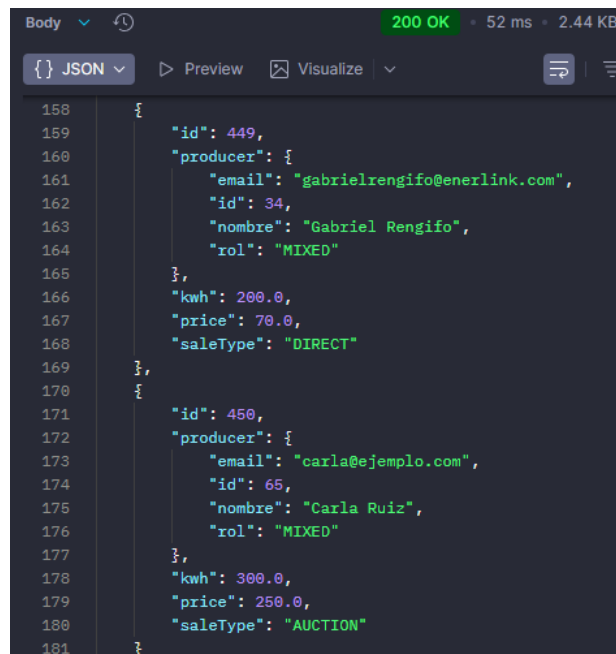
Request

- **Método:** GET
- **URL:** <http://localhost:8080/api/offers>

Resultado esperado

El sistema devuelve una lista con todas las ofertas registradas.

Evidencia



```
Body 200 OK 52 ms 2.44 KB
[{} JSON] Preview Visualize
158 {
159   "id": 449,
160   "producer": {
161     "email": "gabrielrengifo@enerlink.com",
162     "id": 34,
163     "nombre": "Gabriel Rengifo",
164     "rol": "MIXED"
165   },
166   "kwh": 200.0,
167   "price": 70.0,
168   "saleType": "DIRECT"
169 },
170 {
171   "id": 450,
172   "producer": {
173     "email": "carla@ejemplo.com",
174     "id": 65,
175     "nombre": "Carla Ruiz",
176     "rol": "MIXED"
177   },
178   "kwh": 300.0,
179   "price": 250.0,
180   "saleType": "AUCTION"
181 }
```

Consulta de oferta por ID

Esta prueba permite verificar que el sistema pueda recuperar una oferta específica.

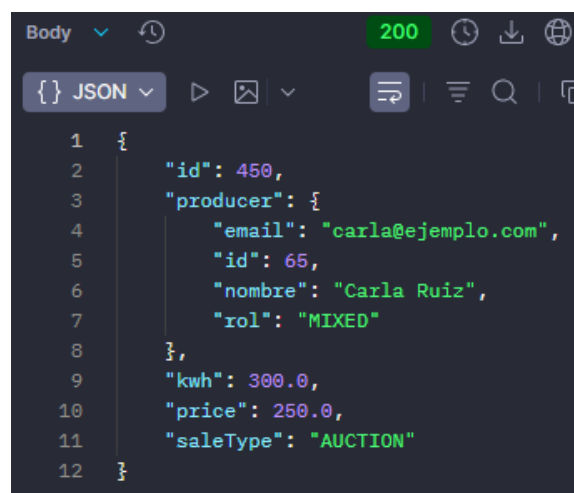
Request

- **Método:** GET
- **URL:** <http://localhost:8080/api/offers/450>

Resultado esperado

El sistema retorna la información de la oferta solicitada.

Evidencia



```
Body 200 OK
[{} JSON]
1 {
2   "id": 450,
3   "producer": {
4     "email": "carla@ejemplo.com",
5     "id": 65,
6     "nombre": "Carla Ruiz",
7     "rol": "MIXED"
8   },
9   "kwh": 300.0,
10  "price": 250.0,
11  "saleType": "AUCTION"
12 }
```


Actualización de oferta

Esta prueba válida la actualización de los datos de una oferta existente.

Request

- **Método:** PUT
- **URL:** <http://localhost:8080/api/offers/450>

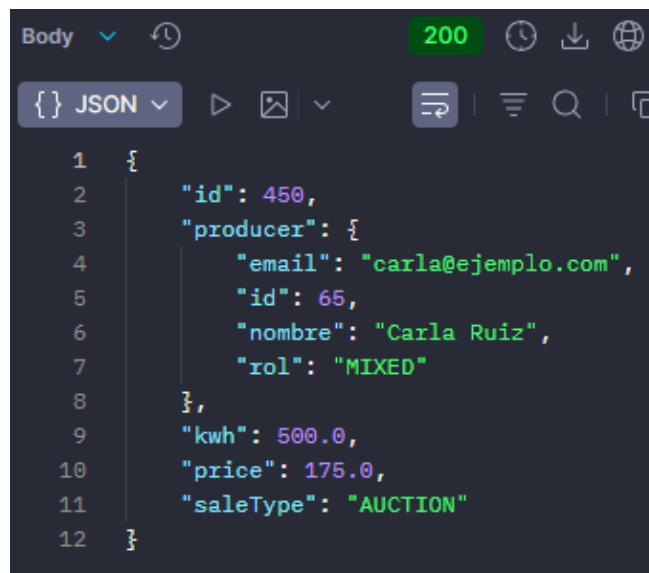
Body

```
{  
  "saleType": "AUCTION",  
  "producerId": 65,  
  "kwh": 500,  
  "price": 175  
}
```

Resultado esperado

La oferta debe actualizar sus valores de energía y precio.

Evidencia



The screenshot shows a REST client interface with a status bar at the top indicating a 200 response. The body is displayed in JSON format. The JSON object contains the following fields: 'id' (450), 'producer' (an object with 'email': 'carla@ejemplo.com', 'id': 65, 'nombre': 'Carla Ruiz', 'rol': 'MIXED'), 'kwh' (500.0), 'price' (175.0), and 'saleType' (AUCTION).

```
1  {  
2    "id": 450,  
3    "producer": {  
4      "email": "carla@ejemplo.com",  
5      "id": 65,  
6      "nombre": "Carla Ruiz",  
7      "rol": "MIXED"  
8    },  
9    "kwh": 500.0,  
10   "price": 175.0,  
11   "saleType": "AUCTION"  
12 }
```

Eliminación de oferta

Esta prueba verifica que el sistema pueda eliminar una oferta existente.

Request

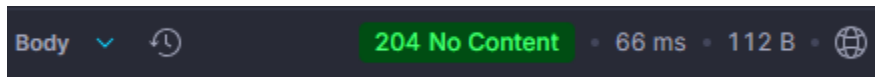
Método: DELETE

URL: <http://localhost:8080/api/offers/450>

Resultado esperado

El sistema elimina la oferta y retorna un código HTTP **204 No Content**.

Evidencia



Resultados de las pruebas

Las pruebas realizadas permitieron verificar que el sistema Enerlink implementa correctamente el patrón Abstract Factory, ya que la creación de ofertas y procesos de venta depende exclusivamente del tipo de venta especificado.

El servicio de aplicación no depende de clases concretas como DirectSaleProcess o AuctionSaleProcess, sino que delega la creación de estos objetos a las fábricas correspondientes.

Esto confirma que el sistema cumple con los principios de diseño asociados al patrón, especialmente:

- Open/Closed Principle
- Dependency Inversion Principle
- Single Responsibility Principle

6.3.5 Justificación de la implementación

Técnicamente:

- La capa de aplicación nunca hace `new DirectSaleProcess(...)` ni `new AuctionSaleProcess(...)`, ni construye directamente `EnergyOffer` acoplada a un `SaleType` concreto.
- La creación de ofertas y procesos de venta se delega a la fábrica correspondiente según el tipo de venta (`DIRECT`, `AUCTION`), garantizando que el servicio no dependa de clases concretas, sino de `EnergySaleFactory` y `SaleProcess`.
- Si se requiere un nuevo tipo de venta, solo hay que agregar una nueva fábrica concreta (por ejemplo, `ScheduledSaleFactory`) e integrarla en la configuración, sin modificar la lógica central del servicio de aplicación.

Desde la perspectiva del negocio en Enerlink:

- Cada tipo de venta puede tener un comportamiento propio (por ejemplo, lógica específica para venta directa o reglas adicionales para subastas), encapsulado en su implementación de `SaleProcess`.

- La fábrica asegura que, para cada tipo de venta, se construya siempre una combinación coherente de EnergyOffer y SaleProcess, garantizando consistencia en la forma en que se gestionan las ofertas energéticas.
- Se reducen errores al centralizar la lógica de creación por tipo de venta y se mejora la capacidad de escalar el sistema a nuevos mecanismos de intercambio energético.

Relación con Clean Architecture y SOLID:

- La capa de aplicación depende de abstracciones (EnergySaleFactory, SaleProcess) y no de clases concretas, aplicando el principio de Inversión de Dependencias.
- Cada fábrica concreta tiene una única responsabilidad: crear la familia de objetos correspondiente a un tipo de venta específico (oferta + proceso de venta), respetando el Single Responsibility Principle.
- El sistema es abierto a extensiones y cerrado a modificaciones, ya que nuevos tipos de venta se añaden mediante nuevas fábricas sin cambiar el código existente de los servicios, cumpliendo el Open/Closed Principle.

6.3.6 Aclaración sobre la implementación actual

Actualmente, las fábricas de venta de energía se enfocan en construir objetos simples para demostrar correctamente el patrón Abstract Factory dentro del contexto de Enerlink, sin incorporar todavía reglas avanzadas de negocio ni cálculos complejos asociados a la transacción o la subasta.

Esta implementación mínima permite validar el diseño del patrón y su integración con la arquitectura hexagonal sin añadir lógica extra, facilitando la comprensión del rol de cada componente. A futuro, si se requiere mejorar el flujo de creación (por ejemplo, inicializar estados específicos de la subasta, aplicar comisiones, configurar parámetros adicionales de la transacción o asociar políticas de validación por tipo de venta), estas responsabilidades podrán incorporarse dentro de las fábricas concretas y de los procesos de venta correspondientes, manteniendo intacta la interfaz EnergySaleFactory y sin romper la estructura actual del patrón.

6.4. PATRÓN FLUENT BUILDER (TRANSACTION)

6.4.1 Descripción del patrón

El patrón **Builder** es un patrón de diseño creacional cuyo objetivo es **separar la construcción de un objeto complejo de su representación**, permitiendo que el mismo proceso de construcción pueda generar diferentes configuraciones del objeto.

Una variante moderna muy utilizada en lenguajes como Java es el **Fluent Builder**, el cual permite construir objetos mediante **encadenamiento de métodos (method chaining)**. Este enfoque mejora la legibilidad del código y facilita la creación de objetos con múltiples atributos.

El patrón resulta especialmente útil cuando una clase:

- Posee múltiples atributos que deben inicializarse.
- Requiere validaciones antes de crear la instancia.
- Debe mantenerse **inmutable** después de su creación.
- Podría requerir múltiples configuraciones dependiendo del contexto.

En el proyecto **Enerlink**, este patrón fue implementado en la clase **Transaction**, la cual representa una transacción de compra y venta de energía entre usuarios dentro de la plataforma.

Dado que una transacción contiene diversos atributos obligatorios como la oferta, el comprador, el vendedor, la cantidad de energía y el precio, el uso de un constructor tradicional con múltiples parámetros podría generar código poco legible y difícil de mantener. Por esta razón se optó por implementar el patrón **Fluent Builder**.

6.4.2 Aplicación del patrón en el proyecto Enerlink

La clase **Transaction** representa una operación comercial dentro del sistema, donde un usuario adquiere cierta cantidad de energía ofrecida por otro usuario.

Esta entidad contiene información relevante como:

- Identificador de la transacción.
- Oferta de energía asociada.
- Usuario comprador.
- Usuario vendedor.
- Cantidad de energía en kilovatios hora (kWh).
- Precio por unidad de energía.
- Fecha y hora de la transacción.

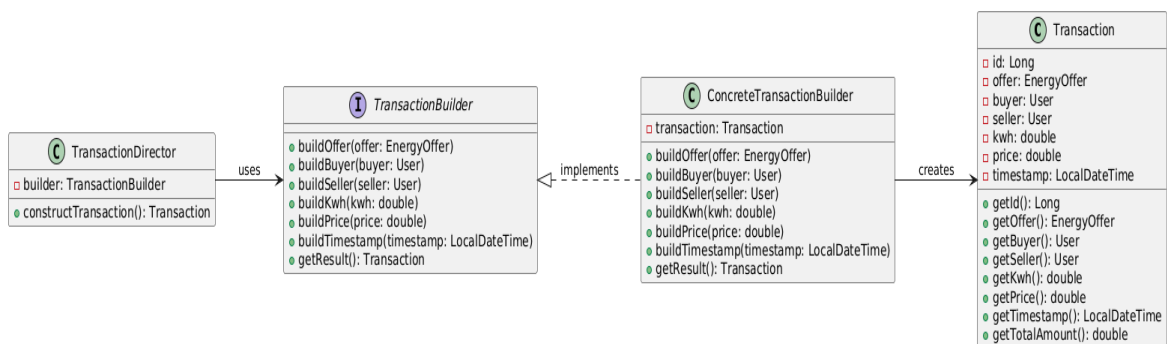
Debido a la cantidad de atributos que conforman la transacción, se decidió implementar el patrón Fluent Builder con el fin de **facilitar la construcción del objeto y mejorar la claridad del código**.

Además, la clase fue diseñada como **inmutable**, lo que significa que una vez creada la transacción, sus valores no pueden ser modificados.

6.4.3 Diagrama UML

En esta sección se presentan dos diagramas UML relacionados con el patrón **Builder** aplicado al contexto del proyecto Enerlink. El primer diagrama corresponde a la **estructura del patrón Builder en su forma clásica**, adaptada al contexto del sistema para la construcción de transacciones energéticas. El segundo diagrama representa la **implementación del Fluent Builder utilizada en el proyecto**, la cual simplifica la estructura del patrón y mejora la legibilidad del código mediante encadenamiento de métodos

UML patrón Builder clasico en Enerlink

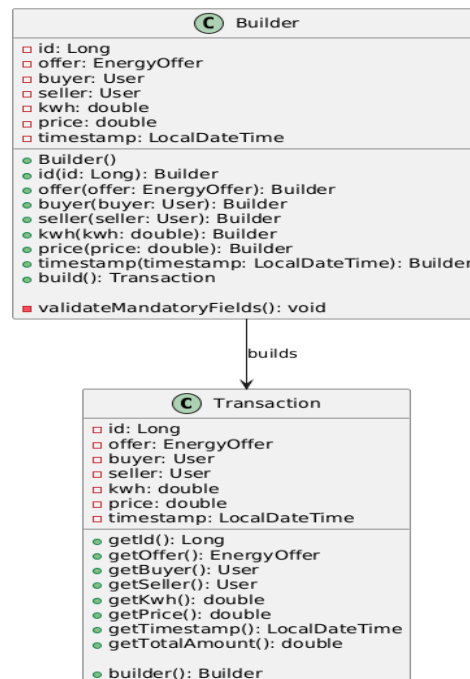


El diagrama UML muestra cómo se estructuraría el patrón **Builder en su versión clásica dentro del proyecto Enerlink** para la construcción de la entidad Transaction. En este enfoque existiría una interfaz Builder encargada de definir los métodos necesarios para construir las diferentes partes de la transacción, mientras que una clase ConcreteBuilder implementaría dichos métodos para crear el objeto Transaction.

Adicionalmente, un componente director sería responsable de coordinar el proceso de construcción del objeto paso a paso, indicando al Builder cómo ensamblar la transacción a partir de sus atributos como la oferta, el comprador, el vendedor, la cantidad de energía y el precio.

Este diseño permite separar el proceso de construcción del objeto Transaction de su representación final, manteniendo organizada la lógica de creación dentro del sistema.

UML Fluent Builder en Enerlink



El diagrama UML de la implementación **Fluent Builder** refleja la forma en que este patrón fue aplicado dentro del proyecto Enerlink para la construcción de la entidad **Transaction**. En este modelo, la clase **Transaction** contiene una **clase interna estática llamada Builder**, encargada de gestionar el proceso de creación del objeto. A diferencia de la estructura clásica, no se utiliza un director, ya que el propio cliente controla la construcción mediante el encadenamiento de métodos del **Builder**.

Cada método del **Builder** retorna la propia instancia del **Builder** (`return this`), lo que permite configurar progresivamente los atributos de la transacción hasta invocar el método `build()`, encargado de validar los datos y generar la instancia final.

Esta versión fue seleccionada porque **reduce la complejidad estructural del patrón, mejora la legibilidad del código y facilita la construcción de objetos con múltiples atributos**, integrándose de forma natural con el diseño del dominio del sistema y permitiendo mantener la entidad **Transaction** como un objeto inmutable una vez creado.

6.4.4 Implementación del Fluent Builder

La implementación del patrón Fluent Builder se realiza mediante una **clase interna estática llamada Builder**, la cual permite construir una instancia de Transaction paso a paso.

El Builder contiene los mismos atributos que la clase principal, pero en versión mutable, permitiendo asignar sus valores mediante métodos configuradores.

Cada método del Builder retorna la propia instancia del Builder (return this), lo que permite el **encadenamiento de métodos**, característica principal del Fluent Builder.

Clase Transaction

```
1 public final class Transaction {
2
3     private final Long id;
4     private final EnergyOffer offer;
5     private final User buyer;
6     private final User seller;
7     private final double kwh;
8     private final double price;
9     private final LocalDateTime timestamp;
10
11     private Transaction(Builder builder) {
12         this.id = builder.id;
13         this.offer = builder.offer;
14         this.buyer = builder.buyer;
15         this.seller = builder.seller;
16         this.kwh = builder.kwh;
17         this.price = builder.price;
18         this.timestamp = builder.timestamp != null ? builder.timestamp : LocalDateTime.now();
19     }
20 }
```

Clase Builder

```
1 public static class Builder {
2     private Long id;
3     private EnergyOffer offer;
4     private User buyer;
5     private User seller;
6     private double kwh;
7     private double price;
8     private LocalDateTime timestamp;
9
10    private Builder() {}
11
12
13    public Builder id(Long id) {
14        this.id = id;
15        return this;
16    }
17
18    public Builder offer(EnergyOffer offer) {
19        this.offer = offer;
20        return this;
21    }
22
23    public Builder buyer(User buyer) {
24        this.buyer = buyer;
25        return this;
26    }
27
28    public Builder seller(User seller) {
29        this.seller = seller;
30        return this;
31    }
32
33    public Builder kwh(double kwh) {
34        this.kwh = kwh;
35        return this;
36    }
37
38    public Builder price(double price) {
39        this.price = price;
40        return this;
41    }
42
43    public Builder timestamp(LocalDateTime timestamp) {
44        this.timestamp = timestamp;
45        return this;
46    }
47 }
```

Una vez configurados los atributos necesarios, se invoca el método build(), el cual se encarga de validar los datos antes de crear la instancia final de Transaction.

Dentro del método build() se ejecuta el método validateMandatoryFields(), el cual verifica que los campos obligatorios tengan valores válidos.

Las validaciones implementadas incluyen:

- Verificar que la oferta (offer) no sea nula.
- Verificar que el comprador (buyer) no sea nulo.
- Verificar que el vendedor (seller) no sea nulo.
- Validar que la cantidad de energía (kwh) sea mayor que cero.
- Validar que el precio (price) sea mayor que cero.


```

private void validateMandatoryFields() {
    Objects.requireNonNull(offer, "Offer is mandatory");
    Objects.requireNonNull(buyer, "Buyer is mandatory");
    Objects.requireNonNull(seller, "Seller is mandatory");

    if (kwh ≤ 0) {
        throw new IllegalArgumentException("KWh must be greater than zero");
    }

    if (price ≤ 0) {
        throw new IllegalArgumentException("Price must be greater than zero");
    }
}

```

Si todas las validaciones se cumplen, el método build() crea una nueva instancia de Transaction.

El patrón también es utilizado dentro de los procesos de venta implementados en el sistema, como **DirectSaleProcess** y **AuctionSaleProcess**, donde se genera la transacción una vez finalizada la operación comercial.

Interfaz SaleProcess

```

1 package com.enerlink.enerlink.energia.dominio.proceso;
2
3 import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;
4 import com.enerlink.enerlink.energia.dominio.modelo.Transaction;
5 import com.enerlink.enerlink.usuario.dominio.modelo.User;
6
7 public interface SaleProcess {
8
9     Transaction execute(EnergyOffer offer, User buyer, double kwh);
10
11     default Transaction execute(EnergyOffer offer, User buyer, double kwh, double customPrice) {
12         throw new UnsupportedOperationException("Custom price not supported for this sale type");
13     }
14 }

```

Implementación DirectProcess

```
1 public class DirectSaleProcess implements SaleProcess {
2
3     @Override
4     public Transaction execute(EnergyOffer offer, User buyer, double kwh) {
5         double price = offer.getPrice();
6
7         Transaction transaction = Transaction.builder()
8             .offer(offer)
9             .buyer(buyer)
10            .seller(offer.getProducer())
11            .kwh(kwh)
12            .price(price)
13            .timestamp(LocalDateTime.now())
14            .build();
15
16        System.out.println("Venta directa completada: " + transaction.getTotalAmount());
17        return transaction;
18    }
```

Implementación AuctionSaleProcess

```
1 public class AuctionSaleProcess implements SaleProcess {
2
3     @Override
4     public Transaction execute(EnergyOffer offer, User buyer, double kwh) {
5         double finalPrice = calculateWinningBid(offer);
6
7         Transaction transaction = Transaction.builder()
8             .offer(offer)
9             .buyer(buyer)
10            .seller(offer.getProducer())
11            .kwh(kwh)
12            .price(finalPrice)
13            .timestamp(LocalDateTime.now())
14            .build();
15
16        System.out.println("Subasta ganada por: " + buyer.getNombre() + " - Precio final: " + transaction.getTotalAmount());
17        return transaction;
18    }
19
20    private double calculateWinningBid(EnergyOffer offer) {
21        return offer.getPrice() * 0.9;
22    }
23 }
```

6.4.5 Justificación de la implementación

Técnicamente:

- La creación de objetos Transaction no se realiza mediante constructores públicos con múltiples parámetros, sino a través del Fluent Builder, lo que permite construir la instancia paso a paso mediante encadenamiento de métodos.
- La lógica de validación de los datos necesarios para crear una transacción (oferta, comprador, vendedor, cantidad de energía y precio) se centraliza dentro del método build() del Builder, evitando que se creen objetos en estados inválidos.
- Si en el futuro se requieren nuevos atributos o validaciones en la entidad Transaction, estos pueden incorporarse dentro del Builder sin afectar el código existente que utiliza el proceso de construcción, manteniendo estabilidad en la interfaz de uso.

Desde la perspectiva del negocio en Enerlink:

- Las transacciones representan operaciones críticas dentro del sistema de intercambio de energía, por lo que es importante garantizar que cada transacción se cree con información válida y consistente.
- El uso del Builder permite asegurar que todos los datos necesarios para registrar una transacción (oferta, comprador, vendedor, energía y precio) se establezcan de forma explícita antes de generar la instancia final.
- Facilita la evolución del sistema si en el futuro se agregan nuevos datos relevantes a las transacciones, como comisiones, estados de pago o información adicional de mercado energético.

Relación con Clean Architecture y SOLID:

- La construcción de la entidad Transaction se encapsula dentro de su propio Builder, separando la responsabilidad de creación del objeto de la lógica de negocio que utiliza la transacción, lo que favorece una arquitectura más limpia y organizada.
- El Builder tiene una única responsabilidad: gestionar la construcción y validación de los objetos Transaction, cumpliendo con el Single Responsibility Principle (SRP).
- El sistema se mantiene abierto a extensiones y cerrado a modificaciones, ya que nuevos atributos o reglas de validación pueden añadirse dentro del Builder sin modificar el código que utiliza la clase Transaction, respetando el Open/Closed Principle (OCP).

6.4.6 Aclaración sobre la implementación actual

Actualmente, la implementación del patrón Fluent Builder en la clase Transaction se enfoca en demostrar de manera clara el proceso de construcción controlada de objetos dentro del dominio de Enerlink, sin incorporar aún reglas avanzadas de negocio ni lógica compleja asociada al procesamiento de transacciones energéticas.

En esta versión, el Builder se utiliza principalmente para encapsular la creación del objeto Transaction, asegurar la inicialización de sus atributos principales y validar condiciones básicas como la existencia de la oferta, el comprador, el vendedor, así como valores válidos para la cantidad de energía (kwh) y el precio.

A futuro, si el sistema requiere ampliar la lógica asociada a las transacciones, el Builder podrá evolucionar para incorporar responsabilidades adicionales durante el proceso de construcción, como por ejemplo la inicialización de estados de la transacción o la incorporación de políticas específicas de validación según el tipo de operación energética.

6.4.7 Pruebas unitarias del Fluent Builder

Con el objetivo de verificar el correcto funcionamiento de la implementación del patrón **Fluent Builder** en la entidad Transaction, se desarrolló un conjunto de pruebas unitarias utilizando **JUnit 5**. Estas pruebas permiten validar que el proceso de construcción del objeto funcione correctamente, que las reglas de validación se ejecuten adecuadamente y que la interfaz fluida del Builder opere según lo esperado.

Aunque se implementaron múltiples pruebas para cubrir diferentes escenarios, a continuación se presentan **cuatro pruebas representativas** que evidencian los aspectos más importantes del patrón Fluent Builder dentro del sistema Enerlink.

6.4.7.1 Construcción de una transacción mediante Fluent Builder

Esta prueba verifica que una instancia de Transaction pueda construirse correctamente utilizando el Builder y que todos los atributos asignados durante el proceso de construcción se almacenen adecuadamente en el objeto final.

Durante la prueba se configuran los atributos principales de la transacción oferta de energía, comprador, vendedor, cantidad de energía, precio y fecha utilizando el encadenamiento de métodos del Builder. Posteriormente se invoca el método build() para generar la instancia final.

El resultado esperado es que el objeto Transaction sea creado correctamente y que cada uno de sus atributos coincida con los valores proporcionados durante el proceso de construcción. Esta prueba demuestra el uso básico del patrón Builder y confirma que el proceso de creación del objeto funciona correctamente.

```

1  @Test
2      @DisplayName("1. Should build Transaction using Fluent Builder")
3      void shouldBuildTransactionUsingFluentBuilder() {
4          // Given: All required fields for Transaction
5          LocalDateTime timestamp = LocalDateTime.of(2024, 6, 15, 10, 30);
6
7          // When: Building Transaction using Fluent Builder pattern
8          Transaction transaction = Transaction.builder()
9              .offer(energyOffer)
10             .buyer(consumer)
11             .seller(producer)
12             .kwh(100)
13             .price(150)
14             .timestamp(timestamp)
15             .build();
16
17             // Then: All fields are correctly assigned
18             assertNotNull(transaction);
19             assertEquals(energyOffer, transaction.getOffer());
20             assertEquals(consumer, transaction.getBuyer());
21             assertEquals(producer, transaction.getSeller());
22             assertEquals(100.0, transaction.getKwh());
23             assertEquals(150.0, transaction.getPrice());
24             assertEquals(timestamp, transaction.getTimestamp());
25     }

```

6.4.7.2 Validación de reglas durante la construcción

Una característica importante de la implementación del Builder en el proyecto es la **validación de los datos obligatorios antes de crear la instancia final**. Esta prueba verifica que el sistema impida la creación de una transacción con valores inválidos.

En este caso se evalúa el escenario en el que la cantidad de energía (kwh) es igual a cero. Durante la ejecución del método build(), el Builder ejecuta las validaciones definidas en validateMandatoryFields(), lo que provoca el lanzamiento de una excepción IllegalArgumentException.

El resultado esperado es que la transacción no se cree y que se genere un mensaje de error indicando que la cantidad de energía debe ser mayor a cero. Esto demuestra que el Builder no solo construye el objeto, sino que también **garantiza la integridad de los datos antes de permitir su creación**.

```

1  @Test
2      @DisplayName("5. Should throw exception when kwh is invalid (zero)")
3      void shouldThrowExceptionWhenKwhIsZero() {
4          // When & Then: Attempting to build with kwh = 0 should throw IllegalArgumentException
5          IllegalArgumentException exception = assertThrows(
6              IllegalArgumentException.class,
7              () -> Transaction.builder()
8                  .offer(energyOffer)
9                  .buyer(consumer)
10                 .seller(producer)
11                 .kwh(0)
12                 .price(150)
13                 .build()
14          );
15
16          assertEquals("KWh must be greater than zero", exception.getMessage());
17      }

```

6.4.7.3 Verificación del comportamiento del objeto construido

Una vez creada la instancia de Transaction, es necesario verificar que sus métodos funcionen correctamente. Esta prueba evalúa el método getTotalAmount(), el cual calcula el valor total de la transacción multiplicando la cantidad de energía (kwh) por el precio unitario (price).

En la prueba se construye una transacción con valores conocidos y posteriormente se verifica que el cálculo del total sea correcto.

El resultado esperado es que el método retorne el valor correspondiente a la multiplicación de ambos atributos, confirmando que el objeto construido mediante el Builder mantiene correctamente su comportamiento dentro del dominio del sistema.

```

1  @Test
2      @DisplayName("9. Should calculate total amount correctly (kwh * price)")
3      void shouldCalculateTotalAmountCorrectly() {
4          // Given: kwh = 100, price = 150
5          Transaction transaction = Transaction.builder()
6              .offer(energyOffer)
7              .buyer(consumer)
8              .seller(producer)
9              .kwh(100)
10             .price(150)
11             .build();
12
13             // Then: Total amount should be 100 * 150 = 15000
14             assertEquals(15000.0, transaction.getTotalAmount(), 0.001);
15     }

```

6.4.7.4 Demostración del encadenamiento de métodos (Fluent Interface)

Esta prueba tiene como objetivo demostrar explícitamente la característica principal del **Fluent Builder**, que es el **encadenamiento de métodos** durante la construcción del objeto.

Cada método retorna la instancia del Builder, permitiendo que múltiples configuraciones se encadenen en una sola expresión. Esto mejora la legibilidad del código y facilita la comprensión del proceso de creación del objeto.

El resultado esperado es que todos los atributos configurados mediante este encadenamiento se asignen correctamente al objeto Transaction, demostrando la principal ventaja del Fluent Builder frente a otros enfoques de construcción de objetos.

```
1  @Test
2      @DisplayName("13. Should demonstrate fluent method chaining")
3      void shouldDemonstrateFluentMethodChaining() {
4          // This test explicitly demonstrates the fluent method chaining pattern
5          // The pattern: .method(value).method(value).method(value).build()
6
7          Transaction transaction = Transaction.builder()
8              .id(1L)
9              .offer(energyOffer)
10             .buyer(consumer)
11             .seller(producer)
12             .kwh(75)
13             .price(200)
14             .timestamp(LocalDateTime.now())
15             .build();
16
17         // Verify fluent chaining works correctly
18         assertEquals("Transaction fields",
19             () -> assertEquals(1L, transaction.getId()),
20             () -> assertEquals(energyOffer, transaction.getOffer()),
21             () -> assertEquals(consumer, transaction.getBuyer()),
22             () -> assertEquals(producer, transaction.getSeller()),
23             () -> assertEquals(75.0, transaction.getKwh()),
24             () -> assertEquals(200.0, transaction.getPrice()),
25             () -> assertNotNull(transaction.getTimestamp())
26         );
27     }
```

6.4.7.5 Ejecución de los test unitarios

```
[INFO] Results:
[INFO]
[INFO] Tests run: 14, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 19.553 s
[INFO] Finished at: 2026-03-12T08:53:10-05:00
[INFO] -----
```

Para validar la correcta implementación del patrón Fluent Builder, se ejecutaron las pruebas unitarias utilizando JUnit 5 dentro del entorno de desarrollo. Todas las pruebas fueron ejecutadas satisfactoriamente, sin presentar fallos ni errores durante la ejecución.

Este resultado confirma que la implementación del Builder funciona correctamente, validando tanto la construcción del objeto Transaction, las reglas de validación implementadas en el método build(), así como el correcto funcionamiento del encadenamiento de métodos característico del patrón Fluent Builder.

6.5. PATRÓN PROTOTYPE (ENERGYOFFER)

6.5.1 Descripción del patrón

El patrón Prototype es un patrón de diseño creacional cuyo objetivo es permitir la creación de nuevos objetos a partir de la clonación de instancias existentes, en lugar de crear nuevas instancias desde cero mediante constructores.

Este patrón resulta especialmente útil cuando:

- La creación de objetos es costosa en términos de recursos o tiempo.
- Se requiere generar múltiples objetos similares con ligeras variaciones.
- Se desea desacoplar la lógica de creación de objetos de sus clases concretas.
- Se busca evitar la duplicación de código al construir objetos complejos.

El patrón Prototype define una interfaz común que declara el método de clonación, el cual es implementado por las clases concretas que desean permitir la copia de sus instancias.

En el proyecto Enerlink, este patrón fue implementado en la clase **EnergyOffer**, la cual representa una oferta de energía dentro del sistema. Esta implementación permite generar nuevas ofertas basadas en otras existentes, facilitando la reutilización y modificación de datos sin afectar la instancia original.

6.5.2 Aplicación del patrón en el proyecto Enerlink

La clase EnergyOffer representa una oferta de energía publicada por un usuario dentro del marketplace de Enerlink.

Esta entidad contiene información relevante como:

- Identificador de la oferta.
- Usuario productor (quien ofrece la energía).
- Cantidad de energía en kilovatios hora (kWh).
- Precio por unidad de energía.
- Tipo de venta (directa o subasta).

Debido a la necesidad de crear múltiples ofertas similares a partir de una existente (por ejemplo, variando el precio, la cantidad o el tipo de venta), se decidió implementar el patrón Prototype.

Para ello, se definió una interfaz genérica `Prototype<T>`, la cual establece el contrato para la clonación de objetos mediante los métodos:

- `clone()` → para realizar copias profundas.
- `shallowClone()` → para realizar copias superficiales.

La clase `EnergyOffer` implementa esta interfaz, proporcionando mecanismos flexibles para duplicar instancias y modificarlas según el contexto del negocio.

6.5.3 Diagrama UML

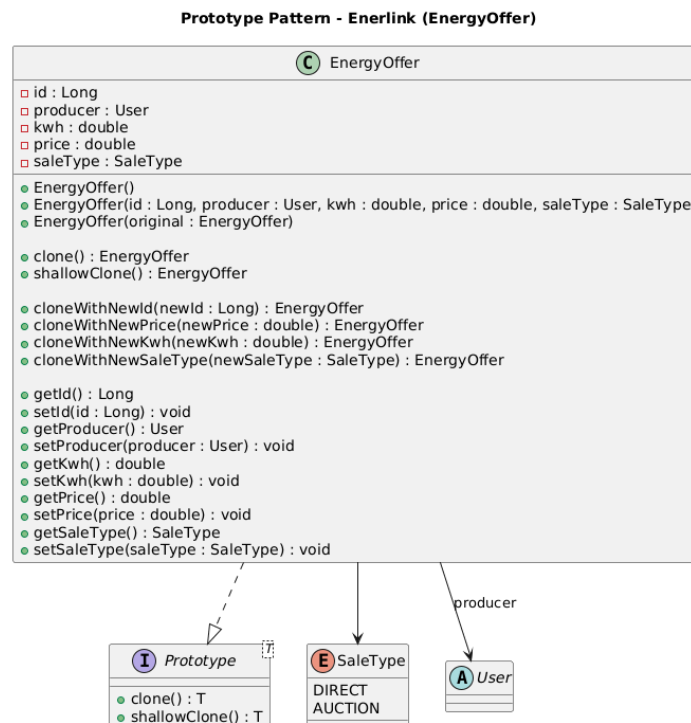
En esta sección se presentan los diagramas UML relacionados con el patrón Prototype aplicado al contexto del proyecto Enerlink.

El diagrama UML muestra cómo se estructura el patrón Prototype dentro del sistema. En este enfoque existe una interfaz `Prototype` que define el método de clonación, el cual es implementado por la clase concreta `EnergyOffer`.

La interfaz `Prototype` actúa como contrato común para todos los objetos que pueden ser clonados, mientras que `EnergyOffer` implementa la lógica específica para copiar sus atributos internos.

A diferencia de otros patrones creacionales como `Builder`, el patrón `Prototype` no requiere un director ni una jerarquía compleja de construcción, ya que la creación de objetos se realiza directamente a partir de instancias existentes.

Este diseño permite desacoplar la lógica de creación de objetos de su representación, facilitando la generación de nuevas instancias sin depender de constructores complejos.



6.5.4 Implementación del Prototype

La implementación del patrón Prototype en el proyecto se realiza mediante la interfaz genérica `Prototype<T>`, la cual define los métodos necesarios para la clonación de objetos.

La clase `EnergyOffer` implementa esta interfaz y proporciona dos tipos de clonación:

- Clonación profunda (`clone`):

Se implementa mediante un constructor de copia que recibe una instancia original y crea una nueva instancia con los mismos valores. Esto permite generar objetos independientes del original.

- Clonación superficial (`shallowClone`):

Se crea una nueva instancia copiando directamente las referencias de los atributos, lo que implica que algunos objetos internos pueden ser compartidos entre la instancia original y la copia.

Adicionalmente, la clase `EnergyOffer` incluye métodos especializados que extienden el uso del patrón Prototype, permitiendo crear clones con modificaciones específicas, tales como:

- `cloneWithNewId()`
- `cloneWithNewPrice()`
- `cloneWithNewKwh()`
- `cloneWithNewSaleType()`

Estos métodos facilitan la creación de nuevas ofertas basadas en una existente, manteniendo la consistencia del modelo y reduciendo la duplicación de código.

Interfaz Prototype

```
1  public interface Prototype<T> {
2
3      /**
4       * Creates a deep copy of this object.
5       *
6       * @return a new instance that is a copy of this object
7       * @throws CloneNotSupportedException if cloning is not supported
8       */
9      T clone();
10
11     /**
12      * Creates a shallow copy of this object.
13      * Shares references to mutable objects with the original.
14      *
15      * @return a new instance with shallow copied references
16      */
17     T shallowClone();
18
19 }
```

Clase EnergyOffer

```
1 public class EnergyOffer implements Prototype<EnergyOffer> {
2
3     private Long id;
4     private User producer;
5     private double kwh;
6     private double price;
7     private SaleType saleType;
8
9
10    @Override
11    public EnergyOffer clone() {
12        return new EnergyOffer(this);
13    }
14
15    @Override
16    public EnergyOffer shallowClone() {
17        if (this == null) {
18            throw new IllegalStateException("Cannot clone null instance");
19        }
20        EnergyOffer copy = new EnergyOffer();
21        copy.id = this.id;
22        copy.producer = this.producer;
23        copy.kwh = this.kwh;
24        copy.price = this.price;
25        copy.saleType = this.saleType;
26        return copy;
27    }
28
29    /**
30     * Creates a clone with a new ID assigned.
31     * Useful for creating new offers based on templates.
32     *
33     * @param newId the ID for the cloned offer
34     * @return a new EnergyOffer with the specified ID
35     */
36    public EnergyOffer cloneWithNewId(Long newId) {
37        EnergyOffer clone = this.clone();
38        clone.setId(newId);
39        return clone;
40    }
41
42    /**
43     * Creates a clone with modified price.
44     *
45     * @param newPrice the new price per kWh
46     * @return a new EnergyOffer with the specified price
47     */
48    public EnergyOffer cloneWithNewPrice(double newPrice) {
49        EnergyOffer clone = this.clone();
50        clone.setPrice(newPrice);
51        return clone;
52    }
53
54    /**
55     * Creates a clone with modified quantity.
56     *
57     * @param newKwh the new amount of energy in kWh
58     * @return a new EnergyOffer with the specified quantity
59     */
60    public EnergyOffer cloneWithNewKwh(double newKwh) {
61        EnergyOffer clone = this.clone();
62        clone.setKwh(newKwh);
63        return clone;
64    }
65
66    /**
67     * Creates a clone with a different sale type.
68     *
69     * @param newSaleType the new sale type
70     * @return a new EnergyOffer with the specified sale type
71     */
72    public EnergyOffer cloneWithNewSaleType(SaleType newSaleType) {
73        EnergyOffer clone = this.clone();
74        clone.setSaleType(newSaleType);
75        return clone;
76    }
77
78 }
79
```

6.5.5 Justificación de la implementación

Técnicamente:

- La creación de objetos EnergyOffer no depende exclusivamente de constructores, sino que puede realizarse mediante clonación, lo que mejora la flexibilidad del sistema.
- La lógica de copia de objetos se centraliza dentro de la propia clase, evitando inconsistencias en la creación de nuevas instancias.
- La implementación de clonación profunda y superficial permite elegir el nivel de independencia requerido entre objetos según el contexto.
- La incorporación de métodos de clonación con modificación permite reutilizar instancias existentes sin alterar su estado original.
- Si en el futuro se agregan nuevos atributos a EnergyOffer, estos pueden incluirse fácilmente en el proceso de clonación sin afectar el código cliente.

Desde la perspectiva del negocio en Enerlink:

- Las ofertas de energía pueden reutilizarse como plantillas para generar nuevas ofertas con ligeras variaciones, optimizando la experiencia del usuario.
- Se evita la modificación accidental de ofertas existentes al trabajar sobre copias independientes.
- Se facilita la gestión de escenarios donde múltiples ofertas comparten características similares, como precios o cantidades ajustadas dinámicamente.
- Permite escalar el sistema de marketplace energético manteniendo consistencia en la creación de datos.

Relación con Clean Architecture y SOLID:

- La lógica de clonación se encuentra encapsulada dentro de la entidad EnergyOffer, lo que mantiene una separación clara de responsabilidades.
- La clase cumple con el **Single Responsibility Principle (SRP)**, ya que gestiona tanto su estado como su proceso de clonación de manera controlada.
- El sistema respeta el **Open/Closed Principle (OCP)**, ya que se pueden extender las capacidades de clonación sin modificar el código existente que utiliza la clase.
- Se reduce el acoplamiento entre componentes, ya que los clientes no necesitan conocer los detalles de construcción de la clase, solo su capacidad de clonarse.

6.5.6 Aclaración sobre la implementación actual

Actualmente, la implementación del patrón Prototype en la clase EnergyOffer se enfoca en proporcionar un mecanismo claro y flexible para la clonación de objetos dentro del dominio de Enerlink, sin incorporar lógica avanzada de negocio asociada a las ofertas de energía.

En esta versión, el patrón se utiliza principalmente para:

- Permitir la duplicación de ofertas existentes.

- Facilitar la modificación controlada de atributos específicos.
- Diferenciar entre clonación profunda y superficial.

Cabe resaltar que, aunque se implementa una clonación tipo “deep copy”, el objeto User (producer) no se clona profundamente, ya que es una entidad abstracta con posibles estructuras complejas. En caso de requerirse una clonación completa de este objeto, sería necesario implementar el patrón Prototype también en dicha clase.

A futuro, esta implementación puede evolucionar para incluir:

- Clonación completa de objetos relacionados.
- Reglas de negocio durante el proceso de clonación.
- Generación automática de identificadores o estados iniciales.

6.5.7 Pruebas unitarias del patrón Prototype

Con el objetivo de verificar el correcto funcionamiento del patrón Prototype en la entidad EnergyOffer, se desarrollaron pruebas unitarias utilizando JUnit 5.

Estas pruebas permiten validar:

- Que la clonación profunda genere objetos independientes del original.
- Que la clonación superficial comparta referencias según lo esperado.
- Que los métodos de clonación con modificación (precio, cantidad, tipo, ID) funcionen correctamente.
- Que no se altere el estado de la instancia original al crear clones.

Aunque se implementaron múltiples pruebas para cubrir diferentes escenarios, se seleccionaron casos representativos que permiten evidenciar el correcto comportamiento del patrón Prototype dentro del sistema Enerlink.

6.5.7.1 Clonación profunda de una oferta de energía

Esta prueba verifica que el método clone() genere correctamente una nueva instancia de **EnergyOffer** a partir de una existente, copiando todos sus atributos principales.

Durante la prueba se invoca el método clone() sobre una oferta original previamente configurada con valores conocidos como identificador, productor, cantidad de energía, precio y tipo de venta.

El resultado esperado es que el objeto clonado no sea nulo y que todos sus atributos coincidan con los de la instancia original. Esta prueba demuestra el funcionamiento básico del patrón Prototype, confirmando que es posible crear nuevas instancias a partir de un objeto existente sin utilizar constructores complejos.

```

1  @Test
2      @DisplayName("1. Should create a deep clone with clone()")
3      void shouldCreateDeepClone() {
4          EnergyOffer cloned = originalOffer.clone();
5
6          assertNotNull(cloned);
7          assertEquals(originalOffer.getId(), cloned.getId());
8          assertEquals(originalOffer.getKwh(), cloned.getKwh(), 0.001);
9          assertEquals(originalOffer.getPrice(), cloned.getPrice(), 0.001);
10         assertEquals(originalOffer.getSaleType(), cloned.getSaleType());
11     }

```

6.5.7.2 Independencia entre la instancia original y el clon

Una característica fundamental del patrón Prototype es que los objetos clonados deben ser independientes del original. Esta prueba verifica que las modificaciones realizadas sobre el clon no afecten el estado de la instancia original.

En la prueba se crea un clon mediante el método `clone()` y posteriormente se modifican sus atributos, como el identificador, la cantidad de energía, el precio y el tipo de venta.

El resultado esperado es que los valores de la oferta original permanezcan sin cambios, mientras que el clon refleje las modificaciones realizadas. Esto demuestra que el proceso de clonación genera una nueva instancia independiente, garantizando la integridad de los datos originales dentro del sistema.

```

1  @Test
2      @DisplayName("2. Clone should be independent from original")
3      void cloneShouldBeIndependent() {
4          EnergyOffer cloned = originalOffer.clone();
5
6          cloned.setId(999L);
7          cloned.setKwh(500.0);
8          cloned.setPrice(0.25);
9          cloned.setSaleType(SaleType.AUCTION);
10
11         assertEquals(1L, originalOffer.getId());
12         assertEquals(100.0, originalOffer.getKwh(), 0.001);
13         assertEquals(0.12, originalOffer.getPrice(), 0.001);
14         assertEquals(SaleType.DIRECT, originalOffer.getSaleType());
15
16         assertEquals(999L, cloned.getId());
17         assertEquals(500.0, cloned.getKwh(), 0.001);
18         assertEquals(0.25, cloned.getPrice(), 0.001);
19         assertEquals(SaleType.AUCTION, cloned.getSaleType());
20     }

```

6.5.7.3 Clonación con modificación de atributos (variaciones del Prototype)

Esta prueba evalúa una extensión del patrón Prototype implementada en la clase **EnergyOffer**, la cual permite crear clones con modificaciones específicas mediante métodos como `cloneWithNewPrice()`.

Durante la prueba se genera un clon a partir de una oferta original, modificando únicamente el precio de la energía, mientras que los demás atributos se mantienen iguales.

El resultado esperado es que el nuevo objeto refleje el cambio en el precio, conservando el resto de los valores originales. Esta prueba demuestra una de las principales ventajas del patrón Prototype en el contexto del negocio, permitiendo generar variaciones de un objeto base de forma sencilla, reutilizando su configuración inicial.

```
1  @Test
2      @DisplayName("11. Should clone with new price")
3      void shouldCloneWithNewPrice() {
4          EnergyOffer cloned = originalOffer.cloneWithNewPrice(0.15);
5
6          assertEquals(1L, cloned.getId());
7          assertEquals(0.15, cloned.getPrice(), 0.001);
8          assertEquals(originalOffer.getKwh(), cloned.getKwh(), 0.001);
9      }
```

6.5.7.4 Diferenciación entre clonación profunda y superficial

Esta prueba tiene como objetivo evidenciar el comportamiento de la clonación en relación con los objetos referenciados, específicamente el atributo `producer`.

En la implementación actual, tanto el método `clone()` como `shallowClone()` comparten la referencia al objeto **User**, lo que implica que no se realiza una clonación profunda de este atributo.

Durante la prueba se verifica que tanto la instancia original como el clon apuntan al mismo objeto productor utilizando comparaciones de referencia.

El resultado esperado es que ambas instancias compartan la misma referencia de `producer`, lo que confirma que la clonación es parcial (deep copy en valores primitivos y shallow copy en objetos complejos). Esta prueba permite entender las implicaciones del diseño actual y resalta una posible mejora futura mediante la implementación del patrón Prototype en la clase **User**.

```

1  @Test
2      @DisplayName("3. Clone should share producer reference (shallow aspect)")
3      void cloneShouldShareProducerReference() {
4          EnergyOffer cloned = originalOffer.clone();
5
6          assertEquals(originalOffer.getProducer(), cloned.getProducer());
7      }

```

6.5.7.5 Ejecución de los test unitarios

```

[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running EnergyOffer Prototype Pattern Tests
[INFO] Running Prototype Interface Implementation Tests
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.106 s -- in Prototype Interface Implementation Tests
[INFO] Running Copy Constructor Tests
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.024 s -- in Copy Constructor Tests
[INFO] Running ToString Tests
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.014 s -- in ToString Tests
[INFO] Running Edge Cases Tests
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.013 s -- in Edge Cases Tests
[INFO] Running Clone Method Tests
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.036 s -- in Clone Method Tests
[INFO] Running Shallow Clone Method Tests
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.017 s -- in Shallow Clone Method Tests
[INFO] Running Clone With Modifications Tests
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.025 s -- in Clone With Modifications Tests
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.281 s -- in EnergyOffer Prototype Pattern Tests
[INFO] Results:
[INFO]
[INFO] Tests run: 19, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.324 s
[INFO] Finished at: 2026-03-26T10:12:20-05:00
[INFO] -----

```

Para validar la correcta implementación del patrón Prototype, se ejecutaron las pruebas unitarias utilizando JUnit 5 dentro del entorno de desarrollo. Todas las pruebas fueron ejecutadas satisfactoriamente, sin presentar fallos ni errores durante la ejecución.

Este resultado confirma que la implementación del patrón Prototype funciona correctamente, validando la clonación de objetos, la independencia entre instancias, la creación de variaciones a partir de objetos existentes y el comportamiento de las referencias compartidas dentro del sistema.

7. PATRONES ESTRUCTURALES

7.1. PATRÓN ADAPTER (IOT)

7.1.1 Descripción del patrón

El patrón Adapter es un patrón de diseño estructural cuyo objetivo es permitir la comunicación entre dos interfaces incompatibles, actuando como un intermediario que transforma la interfaz de una clase en otra que el cliente espera.

Este patrón resulta especialmente útil cuando:

- Se requiere integrar sistemas externos con diferentes formatos de datos.
- Existen múltiples proveedores que exponen información de manera distinta.
- Se desea desacoplar el sistema interno de dependencias externas.
- Se busca mantener una interfaz común dentro del dominio del sistema.

El patrón Adapter define una clase intermediaria que traduce las solicitudes del cliente hacia una interfaz compatible con el sistema, permitiendo reutilizar código existente sin modificar su estructura.

En el proyecto Enerlink, este patrón fue implementado en el módulo IoT, el cual se encarga de integrar diferentes proveedores externos de dispositivos inteligentes. Debido a que cada proveedor maneja estructuras de datos distintas, se implementaron adapters que permiten transformar dichos formatos en un modelo común del sistema, representado por la clase IoTDeviceData.

7.1.2 Aplicación del patrón en el proyecto Enerlink

El módulo IoT del sistema Enerlink tiene como objetivo gestionar información proveniente de dispositivos inteligentes, tales como medidores de energía, paneles solares y sistemas de almacenamiento.

Dichos dispositivos pueden pertenecer a diferentes proveedores externos, entre los cuales se encuentran:

- EnergyCloud
- SmartHome

Cada uno de estos proveedores expone sus datos mediante estructuras propias, lo que genera incompatibilidad con el modelo interno del sistema.

Para resolver este problema, se implementó el patrón Adapter, permitiendo transformar los datos externos a un formato estándar definido por la clase IoTDeviceData.

Para ello, se definió una interfaz llamada IoTDataPort, la cual establece el contrato que deben cumplir todos los adapters. Esta interfaz define métodos para la obtención de datos de dispositivos, tales como:

- fetchDeviceData()
- fetchAllDevices()
- fetchDevicesByLocation()

- fetchDevicesByType()

Las clases EnergyCloudAdapter y SmartHomeAdapter implementan esta interfaz, encargándose de:

- Obtener los datos desde el proveedor externo mediante un cliente específico.
- Convertir los datos crudos (DTO) a un modelo interno utilizando un mapper.
- Retornar la información en el formato IoTDeviceData, comprensible por el sistema.

De esta manera, el sistema puede trabajar con múltiples proveedores sin conocer sus detalles de implementación, garantizando un alto nivel de desacoplamiento.

7.1.3 Diagrama UML

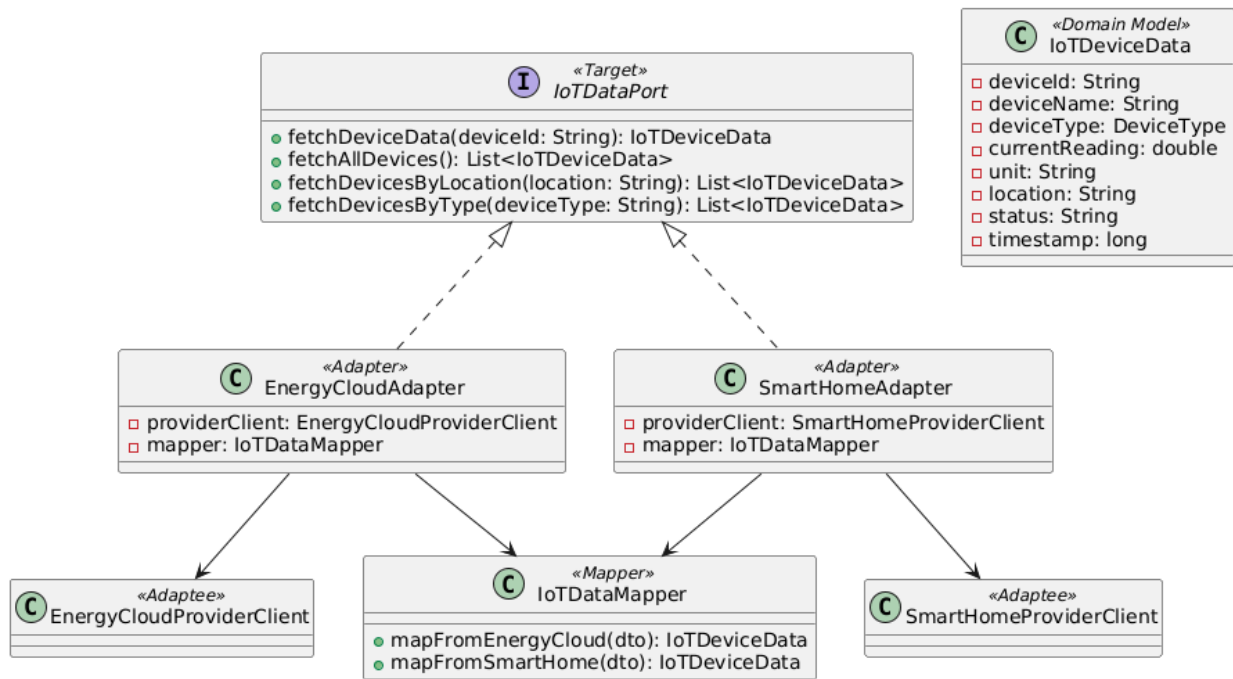
En esta sección se presentan los diagramas UML relacionados con el patrón Adapter aplicado al módulo IoT del proyecto Enerlink.

El diagrama UML muestra cómo se estructura el patrón Adapter dentro del sistema. En este enfoque existe una interfaz IoTDataPort que actúa como contrato (Target), la cual es implementada por las clases concretas EnergyCloudAdapter y SmartHomeAdapter.

Estas clases funcionan como adapters, permitiendo adaptar las interfaces de los proveedores externos (Adaptees), representados por EnergyCloudProviderClient y SmartHomeProviderClient.

Los proveedores externos retornan datos en formatos específicos (DTOs), tales como EnergyCloudProviderDTO y SmartHomeProviderDTO, los cuales son transformados mediante la clase IoTDataMapper hacia el modelo interno IoTDeviceData.

Adapter Pattern - IoT Module (Con Roles Claros)



7.1.4 Implementación del Adapter

La implementación del patrón Adapter en el proyecto se basa en la definición de un puerto dentro del dominio, el cual actúa como interfaz objetivo para la obtención de datos de dispositivos IoT.

La interfaz IoTDataPort define los métodos necesarios para interactuar con los dispositivos, sin especificar cómo se obtienen los datos.

Las clases concretas que implementan este contrato son:

- EnergyCloudAdapter
- SmartHomeAdapter

Cada adapter encapsula la lógica necesaria para interactuar con su respectivo proveedor externo, utilizando un cliente específico:

- EnergyCloudProviderClient
- SmartHomeProviderClient

Estos clientes representan la forma en que se consumen los datos desde cada proveedor.

La transformación de estos datos se realiza mediante la clase IoTDataMapper, la cual se encarga de convertir los DTO en instancias del modelo interno IoTDeviceData.

El flujo de funcionamiento del patrón es el siguiente:

1. El sistema solicita información a través de la interfaz IoTDataPort.
2. El adapter correspondiente invoca al proveedor externo mediante su cliente.
3. Se reciben los datos en formato DTO.
4. El mapper transforma los datos al modelo interno IoTDeviceData.
5. El adapter retorna la información ya adaptada al sistema.

Este proceso garantiza que el dominio del sistema solo interactúe con su propio modelo de datos, sin depender de estructuras externas.

Clase IoTDeviceData

```
1 public class IoTDeviceData {
2
3     private String deviceId;
4     private String deviceName;
5     private DeviceType deviceType;
6     private double currentReading;
7     private String unit;
8     private String location;
9     private String status;
10    private long timestamp;
11
12    public IoTDeviceData() {
13    }
14
15    public IoTDeviceData(String deviceId, String deviceName, DeviceType deviceType,
16                        double currentReading, String unit, String location,
17                        String status, long timestamp) {
18        this.deviceId = deviceId;
19        this.deviceName = deviceName;
20        this.deviceType = deviceType;
21        this.currentReading = currentReading;
22        this.unit = unit;
23        this.location = location;
24        this.status = status;
25        this.timestamp = timestamp;
26    }
```

Interfaz IoTDataPort (Target)

```
1 package com.enerlink.enerlink.iot.dominio.puerto;
2
3 import java.util.List;
4
5 import com.enerlink.enerlink.iot.dominio.modelo.IoTDeviceData;
6
7 public interface IoTDataPort {
8
9     IoTDeviceData fetchDeviceData(String deviceId);
10
11     List<IoTDeviceData> fetchAllDevices();
12
13     List<IoTDeviceData> fetchDevicesByLocation(String location);
14
15     List<IoTDeviceData> fetchDevicesByType(String deviceType);
16 }
```

Class EnergyCloudAdapter (Adapter)

```
1 public class EnergyCloudAdapter implements IoTDataPort {
2
3     private final EnergyCloudProviderClient providerClient;
4     private final IoTDataMapper mapper;
5
6     public EnergyCloudAdapter(EnergyCloudProviderClient providerClient) {
7         this.providerClient = providerClient;
8         this.mapper = new IoTDataMapper();
9     }
10
11     @Override
12     public IoTDeviceData fetchDeviceData(String deviceId) {
13         Optional<Object> rawData = providerClient.fetchDeviceById(deviceId);
14
15         if (rawData.isEmpty()) {
16             return null;
17         }
18
19         EnergyCloudProviderDTO dto = convertToEnergyCloudDTO(rawData.get());
20         return mapper.mapFromEnergyCloud(dto);
21     }
22
23     @Override
24     public List<IoTDeviceData> fetchAllDevices() {
25         List<Object> rawDevices = providerClient.fetchAllDevices();
26         List<IoTDeviceData> result = new ArrayList<>();
27
28         for (Object raw : rawDevices) {
29             EnergyCloudProviderDTO dto = convertToEnergyCloudDTO(raw);
30             IoTDeviceData data = mapper.mapFromEnergyCloud(dto);
31             if (data != null) {
32                 result.add(data);
33             }
34         }
35
36         return result;
37     }
38 }
```

Class SmartHomeAdapter (Adapter)

```
1 public class SmartHomeAdapter implements IoTDataPort {
2
3     private static final Logger logger = LoggerFactory.getLogger(SmartHomeAdapter.class);
4
5     private final SmartHomeProviderClient providerClient;
6     private final IoTDataMapper mapper;
7
8     public SmartHomeAdapter(SmartHomeProviderClient providerClient) {
9         this.providerClient = providerClient;
10        this.mapper = new IoTDataMapper();
11    }
12
13    @Override
14    public IoTDeviceData fetchDeviceData(String deviceId) {
15        logger.info("Fetching device data from SmartHome provider for device: {}", deviceId);
16
17        Optional<Object> rawData = providerClient.fetchDeviceById(deviceId);
18
19        if (rawData.isEmpty()) {
20            logger.warn("Device not found: {}", deviceId);
21            return null;
22        }
23
24        SmartHomeProviderDTO dto = convertToSmartHomeDTO(rawData.get());
25        return mapper.mapFromSmartHome(dto);
26    }
27
28    @Override
29    public List<IoTDeviceData> fetchAllDevices() {
30        logger.info("Fetching all devices from SmartHome provider");
31
32        List<Object> rawDevices = providerClient.fetchAllDevices();
33        List<IoTDeviceData> result = new ArrayList<>();
34
35        for (Object raw : rawDevices) {
36            SmartHomeProviderDTO dto = convertToSmartHomeDTO(raw);
37            IoTDeviceData data = mapper.mapFromSmartHome(dto);
38            if (data != null) {
39                result.add(data);
40            }
41        }
42    }
43 }
```

Clase EnergyCloudProviderClient (Adaptee)

```
1 public class EnergyCloudProviderClient implements IoTProviderClient {
2
3     private static final String PROVIDER_NAME = "EnergyCloud";
4
5     @Override
6     public String getProviderName() {
7         return PROVIDER_NAME;
8     }
9
10    @Override
11    public Optional<Object> fetchDeviceById(String deviceId) {
12        EnergyCloudProviderDTO dto = new EnergyCloudProviderDTO();
13
14        EnergyCloudProviderDTO.ResourceData resource = new EnergyCloudProviderDTO.ResourceData();
15        resource.setDeviceId(deviceId);
16        resource.setName("Energy Device " + deviceId);
17        resource.setType("meter");
18        resource.setManufacturer("EnergyCorp");
19        resource.setModel("EC-2000");
20        dto.setResource(resource);
21
22        EnergyCloudProviderDTO.MeasurementData measurement = new EnergyCloudProviderDTO.MeasurementData();
23        measurement.setCurrentValue(2500.75);
24        measurement.setUom("kWh");
25        measurement.setTimestamp(System.currentTimeMillis());
26        EnergyCloudProviderDTO.AccuracyInfo accuracy = new EnergyCloudProviderDTO.AccuracyInfo();
27        accuracy.setPercentage(99.5);
28        accuracy.setMethod("calibrated");
29        measurement.setAccuracy(accuracy);
30        dto.setMeasurement(measurement);
31
32        EnergyCloudProviderDTO.StatusInfo status = new EnergyCloudProviderDTO.StatusInfo();
33        status.setCode("ACTIVE");
34        status.setDescription("Device is operational");
35        status.setActive(true);
36        dto.setStatus(status);
37
38        return Optional.of(dto);
39    }
```

Clase SmartHomeProviderClient (Adaptee)

```
1 public class SmartHomeProviderClient implements IoTProviderClient {
2
3     private static final String PROVIDER_NAME = "SmartHome";
4
5     @Override
6     public String getProviderName() {
7         return PROVIDER_NAME;
8     }
9
10    @Override
11    public Optional<Object> fetchDeviceById(String deviceId) {
12        SmartHomeProviderDTO dto = new SmartHomeProviderDTO();
13        dto.setDeviceId(deviceId);
14        dto.setDeviceName("Smart Device " + deviceId);
15        dto.setDeviceCategory("smart_meter");
16
17        SmartHomeProviderDTO.MetricValue metrics = new SmartHomeProviderDTO.MetricValue();
18        metrics.setMeasurementType("energy_consumption");
19        metrics.setValue(1234.56);
20        metrics.setUnitOfMeasure("kWh");
21        metrics.setQualityScore(0.98);
22        dto.setMetrics(metrics);
23
24        SmartHomeProviderDTO.LocationInfo location = new SmartHomeProviderDTO.LocationInfo();
25        location.setSiteId("SITE-001");
26        location.setZone("Living Room");
27        location.setAddress("123 Main St");
28        dto.setLocation(location);
29
30        dto.setOperationalState("online");
31        dto.setLastUpdated(System.currentTimeMillis());
32
33        return Optional.of(dto);
34    }
```

Clase IoTDataMapper

```
1 public class IoTDataMapper {
2
3     public IoTDeviceData mapFromSmartHome(SmartHomeProviderDTO dto) {
4         if (dto == null) {
5             return null;
6         }
7
8         IoTDeviceData data = new IoTDeviceData();
9         data.setDeviceId(dto.getDeviceId());
10        data.setDeviceName(dto.getDeviceName());
11        data.setDeviceType(mapDeviceCategory(dto.getDeviceCategory()));
12
13        if (dto.getMetrics() != null) {
14            data.setCurrentReading(dto.getMetrics().getValue());
15            data.setUnit(dto.getMetrics().getUnitOfMeasure());
16        }
17
18        if (dto.getLocation() != null) {
19            String location = dto.getLocation().getZone();
20            if (location == null || location.isEmpty()) {
21                location = dto.getLocation().getAddress();
22            }
23            data.setLocation(location);
24        }
25
26        data.setStatus(dto.getOperationalState());
27        data.setTimestamp(dto.getLastUpdated());
28
29        return data;
30    }
31
32    public IoTDeviceData mapFromEnergyCloud(EnergyCloudProviderDTO dto) {
33        if (dto == null) {
34            return null;
35        }
36
37        IoTDeviceData data = new IoTDeviceData();
38
39        if (dto.getResource() != null) {
40            data.setDeviceId(dto.getResource().getId());
41            data.setDeviceName(dto.getResource().getName());
42            data.setDeviceType(mapResourceType(dto.getResource().getType()));
43        }
44
45        if (dto.getMeasurement() != null) {
46            data.setCurrentReading(dto.getMeasurement().getCurrentValue());
47            data.setUnit(dto.getMeasurement().getUom());
48            data.setTimestamp(dto.getMeasurement().getTimestamp());
49        }
50
51        if (dto.getStatus() != null) {
52            data.setStatus(dto.getStatus().isActive() ? "ACTIVE" : "INACTIVE");
53        }
54
55        return data;
56    }
57 }
```


7.1.5 Justificación de la implementación

Técnicamente:

- El patrón Adapter permite integrar múltiples proveedores externos sin modificar el dominio del sistema.
- La lógica de transformación de datos se encuentra encapsulada dentro de los adapters y mappers, evitando duplicación de código.
- Se mantiene una interfaz común (IoTDataPort), lo que simplifica la interacción con diferentes fuentes de datos.
- El sistema es fácilmente extensible, permitiendo agregar nuevos adapters sin afectar el código existente.
- Se reduce el acoplamiento entre el sistema y las APIs externas.

Desde la perspectiva del negocio en Enerlink:

- Se facilita la integración de nuevos proveedores IoT dentro del marketplace energético.
- Se garantiza consistencia en la representación de datos de dispositivos.
- Se permite escalar el sistema sin depender de un proveedor específico.
- Se mejora la interoperabilidad entre diferentes tecnologías de dispositivos inteligentes.

Relación con Clean Architecture y SOLID:

- La implementación respeta la arquitectura hexagonal, separando claramente dominio, aplicación e infraestructura.
- Se cumple el principio de inversión de dependencias (DIP), ya que el dominio depende de abstracciones (puertos) y no de implementaciones concretas.
- Se respeta el principio de abierto/cerrado (OCP), permitiendo extender el sistema con nuevos adapters sin modificar el código existente.
- Se mantiene el principio de responsabilidad única (SRP), ya que cada clase tiene una función específica (adaptar, mapear, consumir datos, etc.).

7.1.6 Aclaración sobre la implementación actual

Actualmente, la implementación del patrón Adapter en el módulo IoT se enfoca en proporcionar un mecanismo claro para la integración de múltiples proveedores externos, sin incluir lógica compleja de negocio dentro del proceso de adaptación.

En esta versión, el patrón se utiliza principalmente para:

- Transformar datos de proveedores externos a un modelo interno común.
- Desacoplar el dominio del sistema de dependencias externas.
- Facilitar la incorporación de nuevos proveedores sin afectar el sistema existente.

Cabe resaltar que los datos obtenidos desde los proveedores se manejan inicialmente como objetos genéricos (Object), los cuales son posteriormente convertidos a DTO específicos dentro de cada adapter.

A futuro, esta implementación puede evolucionar para incluir:

- Manejo tipado más estricto en los clientes de proveedores.
- Implementación de múltiples estrategias de selección de adapters.
- Integración con servicios reales de IoT en lugar de datos simulados.
- Validaciones adicionales y manejo de errores más robusto en la transformación de datos.

7.1.7 Pruebas unitarias del patrón Adapter

Con el objetivo de verificar el correcto funcionamiento del patrón Adapter en el módulo IoT, se desarrollaron pruebas unitarias utilizando JUnit 5 en conjunto con Mockito para simular el comportamiento de los proveedores externos.

Estas pruebas permiten validar:

Que los adapters obtienen correctamente la información desde los proveedores externos.

Que los datos son transformados adecuadamente al modelo interno IoTDeviceData.

Que el sistema es capaz de trabajar con múltiples proveedores sin modificar su lógica interna.

Que se manejen correctamente escenarios donde no existen datos disponibles.

Se seleccionaron casos representativos que permiten evidenciar el comportamiento del patrón Adapter en diferentes escenarios, especialmente en la transformación de datos provenientes de distintos proveedores.

7.1.7.1 Obtención de datos desde EnergyCloud mediante el Adapter

Esta prueba verifica que el adapter EnergyCloudAdapter es capaz de obtener correctamente la información de un dispositivo desde el proveedor externo EnergyCloud y transformarla al modelo interno del sistema.

Durante la prueba se simula la respuesta del proveedor mediante Mockito, retornando un objeto EnergyCloudProviderDTO con valores previamente definidos.

El adapter invoca al cliente del proveedor, recibe los datos en formato externo y los transforma utilizando el mapper hacia una instancia de IoTDeviceData.

El resultado esperado es que el objeto retornado no sea nulo y que sus atributos principales coincidan con los valores definidos en el DTO, confirmando que el proceso de adaptación se realiza correctamente.

```

1  @Test
2  void fetchDeviceData_withValidId_returnsDeviceData() {
3      EnergyCloudProviderDTO dto = createMockEnergyCloudDTO("EC-001", "Primary Meter", "meter", 2500.75, "kWh", true);
4      when(providerClient.fetchDeviceById("EC-001")).thenReturn(Optional.of(dto));
5
6      IoTDeviceData result = adapter.fetchDeviceData("EC-001");
7
8      assertNotNull(result);
9      assertEquals("EC-001", result.getDeviceId());
10     assertEquals("Primary Meter", result.getDeviceName());
11     assertEquals(DeviceType.SMART_METER, result.getDeviceType());
12     assertEquals(2500.75, result.getCurrentReading());
13     assertEquals("kWh", result.getUnit());
14     assertEquals("ACTIVE", result.getStatus());
15 }

```

7.1.7.2 Transformación completa de datos en EnergyCloudAdapter

Esta prueba tiene como objetivo validar que el adapter realiza correctamente la transformación de todos los atributos del proveedor EnergyCloud hacia el modelo interno.

Durante la prueba se define un DTO con valores específicos como identificador, nombre, tipo de dispositivo, medición, unidad y estado.

El adapter procesa estos datos y genera un objeto IoTDeviceData.

El resultado esperado es que todos los atributos del modelo interno correspondan correctamente con los valores del DTO, incluyendo la conversión del tipo de dispositivo y el estado.

Esta prueba demuestra que el adapter no solo actúa como intermediario, sino que también garantiza la correcta interpretación de los datos externos.

```

1  @Test
2  void fetchDeviceData_mapsAllFieldsCorrectly() {
3      EnergyCloudProviderDTO dto = createMockEnergyCloudDTO("EC-TEST", "Solar Array PV", "solar", 1500.25, "kWh", true);
4      when(providerClient.fetchDeviceById("EC-TEST")).thenReturn(Optional.of(dto));
5
6      IoTDeviceData result = adapter.fetchDeviceData("EC-TEST");
7
8      assertNotNull(result);
9      assertEquals("EC-TEST", result.getDeviceId());
10     assertEquals("Solar Array PV", result.getDeviceName());
11     assertEquals(DeviceType.SOLAR_PANEL, result.getDeviceType());
12     assertEquals(1500.25, result.getCurrentReading());
13     assertEquals("kWh", result.getUnit());
14     assertEquals("ACTIVE", result.getStatus());
15     assertTrue(result.getTimestamp() > 0);
16 }

```

7.1.7.3 Obtención de datos desde SmartHome mediante el Adapter

Esta prueba verifica que el adapter SmartHomeAdapter puede obtener información de dispositivos desde el proveedor SmartHome y adaptarla al modelo interno del sistema.

Al igual que en el caso anterior, se simula la respuesta del proveedor mediante un objeto SmartHomeProviderDTO con datos predefinidos.

El adapter utiliza el cliente correspondiente para obtener los datos, los transforma mediante el mapper y retorna un objeto `IoTDeviceData`.

El resultado esperado es que el objeto generado contenga los valores correctos, validando que el patrón Adapter funciona correctamente con múltiples proveedores.

```
1 @Test
2 void fetchDeviceData_withValidId_returnsDeviceData() {
3     SmartHomeProviderDTO dto = createMockSmartHomeDTO("DEV-001", "Smart Thermostat", "thermostat", 22.5, "°C", "online");
4     when(providerClient.fetchDeviceById("DEV-001")).thenReturn(Optional.of(dto));
5
6     IoTDeviceData result = adapter.fetchDeviceData("DEV-001");
7
8     assertNotNull(result);
9     assertEquals("DEV-001", result.getDeviceId());
10    assertEquals("Smart Thermostat", result.getDeviceName());
11    assertEquals(DeviceType.THERMOSTAT, result.getDeviceType());
12    assertEquals(22.5, result.getCurrentReading());
13    assertEquals("°C", result.getUnit());
14    assertEquals("online", result.getStatus());
15 }
```

7.1.7.4 Transformación completa de datos en SmartHomeAdapter

Esta prueba evalúa la correcta transformación de los datos provenientes del proveedor SmartHome, cuyos formatos difieren significativamente de los utilizados por EnergyCloud.

Durante la prueba se construye un DTO que incluye información de métricas, ubicación, estado y tipo de dispositivo.

El adapter procesa estos datos y los convierte al modelo interno `IoTDeviceData`.

El resultado esperado es que todos los atributos del objeto final coincidan con los valores definidos en el DTO, incluyendo la correcta asignación de la ubicación, el tipo de dispositivo y el estado operacional.

Esta prueba evidencia la principal ventaja del patrón Adapter en el sistema Enerlink, permitiendo integrar múltiples proveedores con estructuras distintas sin afectar el modelo interno.

```
1 @Test
2 void fetchDeviceData_mapsAllFieldsCorrectly() {
3     SmartHomeProviderDTO dto = createMockSmartHomeDTO("SH-TEST", "EV Charger", "evcharger", 45.0, "kWh", "charging");
4     when(providerClient.fetchDeviceById("SH-TEST")).thenReturn(Optional.of(dto));
5
6     IoTDeviceData result = adapter.fetchDeviceData("SH-TEST");
7
8     assertNotNull(result);
9     assertEquals("SH-TEST", result.getDeviceId());
10    assertEquals("EV Charger", result.getDeviceName());
11    assertEquals(DeviceType.ELECTRIC_VEHICLE_CHARGER, result.getDeviceType());
12    assertEquals(45.0, result.getCurrentReading());
13    assertEquals("kWh", result.getUnit());
14    assertEquals("charging", result.getStatus());
15    assertEquals("Living Room", result.getLocation());
16    assertTrue(result.getTimestamp() > 0);
17 }
```

7.1.7.5 Ejecución de los test unitarios

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.enerlink.enerlink.iot.infraestructura.adapter.EnergyCloudAdapterTest
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org.mockito/Mockito.html#0.3
WARNING: A Java agent has been loaded dynamically (C:\Users\estadistica\m2\repository\net\bytebuddy\byte-buddy-agent\1.17.8\byte-buddy-agent-1.17.8.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.315 s -- in com.enerlink.enerlink.iot.infraestructura.adapter.EnergyCloudAdapterTest
[INFO] Running com.enerlink.enerlink.iot.infraestructura.adapter.SmartHomeAdapterTest
14:57:32.670 [main] INFO com.enerlink.enerlink.iot.infraestructura.adapter.SmartHomeAdapter -- Fetching devices by type from SmartHome provider: nonexistent
14:57:32.684 [main] INFO com.enerlink.enerlink.iot.infraestructura.adapter.SmartHomeAdapter -- Fetching devices by type from SmartHome provider: switch
14:57:32.689 [main] INFO com.enerlink.enerlink.iot.infraestructura.adapter.SmartHomeAdapter -- Fetching all devices from SmartHome provider
14:57:32.694 [main] INFO com.enerlink.enerlink.iot.infraestructura.adapter.SmartHomeAdapter -- Fetching device data from SmartHome provider for device: null
14:57:32.695 [main] WARN com.enerlink.enerlink.iot.infraestructura.adapter.SmartHomeAdapter -- Device not found: null
14:57:32.700 [main] INFO com.enerlink.enerlink.iot.infraestructura.adapter.SmartHomeAdapter -- Fetching device data from SmartHome provider for device: SH-TEST
14:57:32.704 [main] INFO com.enerlink.enerlink.iot.infraestructura.adapter.SmartHomeAdapter -- Fetching all devices from SmartHome provider
14:57:32.709 [main] INFO com.enerlink.enerlink.iot.infraestructura.adapter.SmartHomeAdapter -- Fetching device data from SmartHome provider for device: DEV-001
14:57:32.716 [main] INFO com.enerlink.enerlink.iot.infraestructura.adapter.SmartHomeAdapter -- Fetching device data from SmartHome provider for device: NONEXISTENT
14:57:32.716 [main] WARN com.enerlink.enerlink.iot.infraestructura.adapter.SmartHomeAdapter -- Device not found: NONEXISTENT
14:57:32.720 [main] INFO com.enerlink.enerlink.iot.infraestructura.adapter.SmartHomeAdapter -- Fetching devices by location from SmartHome provider: Living Room
14:57:32.725 [main] INFO com.enerlink.enerlink.iot.infraestructura.adapter.SmartHomeAdapter -- Fetching devices by location from SmartHome provider: NonExistent
[INFO] Tests run: 19, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.264 s -- in com.enerlink.enerlink.iot.infraestructura.adapter.SmartHomeAdapterTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 19, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.948 s
[INFO] Finished at: 2026-03-26T14:57:32-05:00
[INFO] -----
```

Para validar la correcta implementación del patrón Adapter, se ejecutaron 19 pruebas unitarias dentro del entorno de desarrollo utilizando JUnit 5.

Todas las pruebas fueron ejecutadas satisfactoriamente, sin presentar fallos ni errores durante su ejecución.

Este resultado confirma que la implementación del patrón Adapter funciona correctamente, garantizando la integración de múltiples proveedores externos, la correcta transformación de datos y el desacoplamiento entre el dominio del sistema y las fuentes externas de información.

7.2. PATRÓN BRIDGE (IOT)

7.2.1 Descripción del patrón

El **patrón Bridge** es un patrón de diseño estructural cuyo objetivo es **desacoplar una abstracción de su implementación**, permitiendo que ambas evolucionen de manera independiente.

Se utiliza especialmente cuando:

- Existen múltiples variantes de **comportamiento** que deben aplicarse sobre diferentes **implementaciones**.
- Se desea **evitar la explosión de clases** al combinar múltiples extensiones de abstracciones con múltiples implementaciones.
- Se busca mantener **flexibilidad y escalabilidad** sin modificar las clases existentes.
- Se pretende **separar responsabilidades**, facilitando pruebas y reutilización.

En el proyecto Enerlink, el patrón Bridge se aplicó en el módulo IoT para separar:

- La **lógica de procesamiento** de los datos IoT (qué se hace con los datos)
- La **fuentes de datos** (de dónde provienen los datos)

Esto permitió combinar distintos tipos de procesadores con cualquier proveedor de datos de manera flexible.

7.2.2 Aplicación del patrón en el proyecto Enerlink

El módulo IoT gestiona información proveniente de dispositivos inteligentes, tales como medidores de energía, paneles solares y sistemas de almacenamiento.

Antes de aplicar Bridge, cada adaptador combinaba **obtención y procesamiento**, generando:

- Duplicación de código
- Acoplamiento fuerte
- Dificultad para añadir nuevos tipos de procesamiento o proveedores

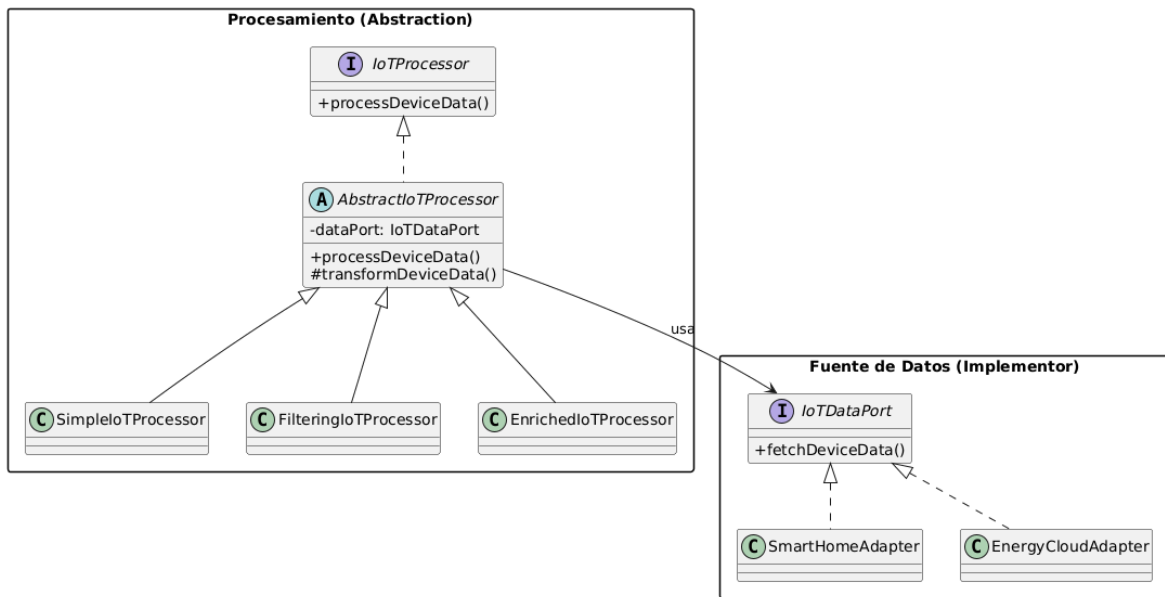
Para resolver esto, se implementó Bridge definiendo:

1. **Abstracción (Abstraction)**: la interfaz IoTProcessor que declara operaciones de procesamiento (processDeviceData, processAllDevices, etc.).
2. **Refined Abstraction**: implementaciones concretas de procesamiento:
 - SimpleIoTProcessor
 - FilteringIoTProcessor
 - EnrichedIoTProcessor
3. **Implementor**: la interfaz IoTDataPort, que define cómo obtener los datos.
4. **Concrete Implementor**: adaptadores existentes que interactúan con proveedores externos:
 - SmartHomeAdapter
 - EnergyCloudAdapter

Gracias a esta separación, cualquier procesador puede trabajar con cualquier fuente de datos sin necesidad de crear combinaciones específicas de clases.

7.2.3 Diagrama UML

El diagrama UML representa de manera simplificada la implementación del patrón Bridge en el módulo IoT de Enerlink:



7.2.4 Implementación del Bridge

Interfaz IoTProcessor

```

1  public interface IoTProcessor {
2
3      IoTDeviceData processDeviceData(String deviceId);
4
5      List<IoTDeviceData> processAllDevices();
6
7      List<IoTDeviceData> processDevicesByLocation(String location);
8
9      List<IoTDeviceData> processDevicesByType(String deviceType);
10 }
```

Clase Abstracta IoTProcessor

```
1 public abstract class AbstractIoTProcessor implements IoTProcessor {
2
3     protected final Logger logger = LoggerFactory.getLogger(getClass());
4
5     protected final IoTDataPort dataPort;
6
7     protected AbstractIoTProcessor(IoTDataPort dataPort) {
8         this.dataPort = dataPort;
9     }
10
11     @Override
12     public IoTDeviceData processDeviceData(String deviceId) {
13         logger.info("Processing device data for: {}", deviceId);
14         IoTDeviceData data = dataPort.fetchDeviceData(deviceId);
15         return transformDeviceData(data);
16     }
17
18     @Override
19     public List<IoTDeviceData> processAllDevices() {
20         logger.info("Processing all devices");
21         List<IoTDeviceData> devices = dataPort.fetchAllDevices();
22         return transformDeviceList(devices);
23     }
24
25     @Override
26     public List<IoTDeviceData> processDevicesByLocation(String location) {
27         logger.info("Processing devices by location: {}", location);
28         List<IoTDeviceData> devices = dataPort.fetchDevicesByLocation(location);
29         return transformDeviceList(devices);
30     }
31 }
```

Clase concreta SimpleIoTProcessor

```
1 public class SimpleIoTProcessor extends AbstractIoTProcessor {
2
3     public SimpleIoTProcessor(IoTDataPort dataPort) {
4         super(dataPort);
5     }
6
7     @Override
8     protected IoTDeviceData transformDeviceData(IoTDeviceData data) {
9         if (data == null) {
10             return null;
11         }
12         logger.debug("Simple processor: returning raw data for device: {}", data.getDeviceId());
13         return data;
14     }
15 }
```


Clase concreta FilteringIoTProcessor

```
1 public class FilteringIoTProcessor extends AbstractIoTProcessor {
2
3     private static final String OFFLINE_STATUS = "offline";
4
5     public FilteringIoTProcessor(IoTDataPort dataPort) {
6         super(dataPort);
7     }
8
9     @Override
10    protected IoTDeviceData transformDeviceData(IoTDeviceData data) {
11        if (data == null) {
12            return null;
13        }
14
15        if (OFFLINE_STATUS.equalsIgnoreCase(data.getStatus())) {
16            logger.info("Filtering out device {} due to offline status", data.getDeviceId());
17            return null;
18        }
19
20        logger.debug("Filtering processor: device {} passed status filter", data.getDeviceId());
21        return data;
22    }
23 }
```

Clase concreta EnrichedIoTProcessor

```
1 public class EnrichedIoTProcessor extends AbstractIoTProcessor {
2
3     private final String enrichmentSource;
4
5     public EnrichedIoTProcessor(IoTDataPort dataPort, String enrichmentSource) {
6         super(dataPort);
7         this.enrichmentSource = enrichmentSource != null ? enrichmentSource : "default";
8     }
9
10    @Override
11    protected IoTDeviceData transformDeviceData(IoTDeviceData data) {
12        if (data == null) {
13            return null;
14        }
15
16        data.setDeviceName(enrichDeviceName(data.getDeviceName(), data.getDeviceId()));
17        data.setTimestamp(Instant.now().toEpochMilli());
18
19        logger.debug("Enriched processor: device {} enriched from source: {}",
20            data.getDeviceId(), enrichmentSource);
21
22        return data;
23    }
24
25    private String enrichDeviceName(String originalName, String deviceId) {
26        String baseName = (originalName == null || originalName.isBlank())
27            ? "Device-" + deviceId
28            : originalName;
29        return "[" + enrichmentSource + "] " + baseName;
30    }
31 }
```

7.2.6 Justificación de la implementación

Técnicamente:

- Permite desacoplar procesamiento y obtención de datos.
- Facilita la creación de nuevos procesadores o adaptadores sin modificar clases existentes.
- Incrementa la reutilización de código y la testabilidad.
- Mantiene coherencia con la arquitectura hexagonal y los principios SOLID.

Desde la perspectiva de negocio:

- Facilita la integración de nuevos proveedores IoT.
- Escalabilidad para agregar nuevos tipos de procesamiento.
- Mejora la interoperabilidad entre dispositivos inteligentes.
- Permite adaptarse a cambios sin afectar el núcleo del sistema.

Relación con Clean Architecture y SOLID:

- La implementación respeta la arquitectura hexagonal, separando claramente dominio, aplicación e infraestructura.
- Se cumple el principio de inversión de dependencias (DIP), ya que el dominio depende de abstracciones y no de implementaciones concretas.
- Se respeta el principio de abierto/cerrado (OCP), permitiendo extender el sistema con nuevos adapters sin modificar el código existente.
- Se mantiene el principio de responsabilidad única (SRP), ya que cada clase tiene una función específica.

7.2.7 Aclaración sobre la implementación actual

Actualmente, la implementación del patrón Bridge en el módulo IoT se centra en:

- Separar responsabilidades entre **procesamiento** y **fuentes de datos**.
- Permitir combinaciones dinámicas de procesadores y adaptadores.
- Mantener compatibilidad con la arquitectura hexagonal.

El Bridge no incluye lógica de negocio compleja dentro del procesamiento, enfocándose en garantizar **flexibilidad y desacoplamiento**.

7.2.8 de pruebas unitarias

Con el objetivo de validar la correcta implementación del patrón Bridge en el módulo IoT, se desarrollaron pruebas unitarias utilizando **JUnit 5** en conjunto con **Mockito**, permitiendo simular el comportamiento de las fuentes de datos (IoTDataPort).

Estas pruebas permiten verificar:

- La correcta **separación entre la lógica de procesamiento y la fuente de datos**.
- La adecuada **delegación de responsabilidades** desde los procesadores hacia el puerto de datos.
- La posibilidad de utilizar **diferentes estrategias de procesamiento** sobre una misma fuente de datos.
- La capacidad del sistema para **extender comportamientos sin modificar el código existente**.

A continuación, se presentan los casos de prueba más representativos que evidencian el correcto funcionamiento del patrón Bridge en el sistema.

7.2.8.1 Uso de múltiples procesadores con la misma fuente de datos

Esta prueba valida que diferentes implementaciones de IoTProcessor pueden trabajar de manera independiente utilizando la misma instancia de IoTDataPort.

Durante la ejecución, se crea un dispositivo en estado *offline* y se procesa utilizando tres tipos de procesadores:

- SimpleIoTProcessor
- FilteringIoTProcessor
- EnrichedIoTProcessor

El resultado esperado es:

- El procesador simple retorna los datos sin modificaciones.
- El procesador de filtrado descarta el dispositivo por estar en estado *offline*.
- El procesador enriquecido retorna el dispositivo agregando información adicional.

Esta prueba demuestra la principal ventaja del patrón Bridge, permitiendo combinar diferentes comportamientos con una misma fuente de datos sin generar acoplamiento entre ellos.

```

1  @Test
2      @DisplayName("Should use different processors with same data port")
3      void differentProcessors_sameDataPort_shouldWorkIndependently() {
4          IoTDeviceData mockData = createTestDevice("DEV-001", "Smart Meter", "offline");
5          when(mockDataPort.fetchDeviceData("DEV-001")).thenReturn(mockData);
6
7          IoTProcessor simpleProcessor = new SimpleIoTProcessor(mockDataPort);
8          IoTProcessor filteringProcessor = new FilteringIoTProcessor(mockDataPort);
9          IoTProcessor enrichedProcessor = new EnrichedIoTProcessor(mockDataPort, "integration");
10
11         IoTDeviceData simpleResult = simpleProcessor.processDeviceData("DEV-001");
12         IoTDeviceData filteringResult = filteringProcessor.processDeviceData("DEV-001");
13         IoTDeviceData enrichedResult = enrichedProcessor.processDeviceData("DEV-001");
14
15         assertNotNull(simpleResult);
16         assertNull(filteringResult);
17         assertNotNull(enrichedResult);
18         assertTrue(enrichedResult.getDeviceName().startsWith("[integration]"));
19     }

```

7.2.8.2 Delegación de responsabilidades hacia el IoTDataPort

Esta prueba verifica que los procesadores no obtienen los datos directamente, sino que delegan esta responsabilidad al IoTDataPort.

Durante la ejecución, se invocan distintos métodos del procesador:

- processDeviceData
- processAllDevices
- processDevicesByLocation
- processDevicesByType

Mediante el uso de Mockito, se valida que cada uno de estos métodos invoca correctamente su equivalente en el IoTDataPort.

El resultado esperado es que todas las llamadas sean delegadas correctamente al puerto de datos, confirmando el uso de composición como mecanismo de conexión entre la abstracción y la implementación.

Esta prueba evidencia el funcionamiento del “puente” del patrón Bridge.

```

1  @Test
2      @DisplayName("Should delegate correctly to IoTDataPort")
3      void processor_shouldDelegateToDataPort() {
4          IoTProcessor processor = new SimpleIoTProcessor(mockDataPort);
5          IoTDeviceData mockData = createTestDevice("DEL-001", "Delegated Device", "online");
6
7          when(mockDataPort.fetchDeviceData("DEL-001")).thenReturn(mockData);
8          when(mockDataPort.fetchAllDevices()).thenReturn(List.of(mockData));
9          when(mockDataPort.fetchDevicesByLocation("TestLocation")).thenReturn(List.of(mockData));
10         when(mockDataPort.fetchDevicesByType("meter")).thenReturn(List.of(mockData));
11
12         processor.processDeviceData("DEL-001");
13         processor.processAllDevices();
14         processor.processDevicesByLocation("TestLocation");
15         processor.processDevicesByType("meter");
16
17         verify(mockDataPort).fetchDeviceData("DEL-001");
18         verify(mockDataPort).fetchAllDevices();
19         verify(mockDataPort).fetchDevicesByLocation("TestLocation");
20         verify(mockDataPort).fetchDevicesByType("meter");
21     }

```

7.2.8.3 Filtrado de dispositivos en FilteringIoTProcessor

Esta prueba tiene como objetivo validar el comportamiento específico del FilteringIoTProcessor, el cual se encarga de filtrar dispositivos según su estado.

Durante la ejecución, se simula un dispositivo con estado *offline*.

El resultado esperado es que el procesador retorne un valor nulo, indicando que el dispositivo ha sido filtrado.

Esta prueba demuestra que es posible modificar el comportamiento del procesamiento sin afectar la forma en que se obtienen los datos, evidenciando la independencia entre ambas jerarquías del patrón Bridge.

```
1  @Test
2  @DisplayName("Should filter out offline devices")
3  void processDeviceData_withOfflineDevice_shouldReturnNull() {
4      IoTDeviceData mockData = createTestDevice("DEV-001", "Smart Meter", "offline");
5      when(mockDataPort.fetchDeviceData("DEV-001")).thenReturn(mockData);
6
7      IoTDeviceData result = processor.processDeviceData("DEV-001");
8
9      assertNull(result);
10 }
```

7.2.8.4 Enriquecimiento de datos en EnrichedIoTProcessor

Esta prueba valida que el EnrichedIoTProcessor es capaz de agregar información adicional a los datos de los dispositivos.

Durante la ejecución, se procesa un dispositivo en estado *online* y se verifica que el nombre del dispositivo es modificado agregando un prefijo correspondiente a la fuente de enriquecimiento.

El resultado esperado es que el nombre del dispositivo contenga el prefijo definido, sin alterar la información original relevante.

Esta prueba demuestra la capacidad del sistema para extender la lógica de procesamiento sin modificar otras clases, cumpliendo con el principio de abierto/cerrado (OCP).

```
1  @Test
2  @DisplayName("Should enrich device name with source prefix")
3  void processDeviceData_shouldEnrichDeviceName() {
4      IoTDeviceData mockData = createTestDevice("DEV-001", "Smart Meter", "online");
5      when(mockDataPort.fetchDeviceData("DEV-001")).thenReturn(mockData);
6
7      IoTDeviceData result = processor.processDeviceData("DEV-001");
8
9      assertNotNull(result);
10     assertTrue(result.getDeviceName().startsWith("[test-source]"));
11     assertTrue(result.getDeviceName().contains("Smart Meter"));
12 }
```

7.2.8.5 Ejecución de los test unitarios

Para validar la implementación del patrón Bridge, se ejecutaron las pruebas unitarias dentro del entorno de desarrollo utilizando **JUnit 5**.

Los resultados obtenidos fueron satisfactorios, evidenciando que:

- La lógica de procesamiento funciona correctamente de manera independiente.
- La comunicación entre procesadores y fuentes de datos se realiza mediante el IoTDataPort.
- Es posible combinar múltiples procesadores con una misma fuente de datos sin generar conflictos.
- El sistema permite extender comportamientos sin afectar la estructura existente.

Estos resultados confirman que la implementación del patrón Bridge en el módulo IoT cumple con los principios de desacoplamiento, flexibilidad y escalabilidad esperados.

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.066 s -- in Bridge Pattern Integration Tests
[INFO] Running SimpleIoTProcessor Tests
14:50:22.262 [main] INFO com.enerlink.enerlink.iot.dominio.servicio.SimpleIoTProcessor -- Processing device data for: DEV-001
14:50:22.268 [main] INFO com.enerlink.enerlink.iot.dominio.servicio.SimpleIoTProcessor -- Processing devices by type: smart_meter
14:50:22.272 [main] INFO com.enerlink.enerlink.iot.dominio.servicio.SimpleIoTProcessor -- Processing all devices
14:50:22.280 [main] INFO com.enerlink.enerlink.iot.dominio.servicio.SimpleIoTProcessor -- Processing device data for: DEV-001
14:50:22.285 [main] INFO com.enerlink.enerlink.iot.dominio.servicio.SimpleIoTProcessor -- Processing all devices
14:50:22.291 [main] INFO com.enerlink.enerlink.iot.dominio.servicio.SimpleIoTProcessor -- Processing devices by location: Living Room
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.036 s -- in SimpleIoTProcessor Tests
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.504 s -- in Bridge Pattern Tests - IoTProcessor
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 22, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.382 s
[INFO] Finished at: 2026-04-08T14:50:22-05:00
[INFO] -----
```

7.3. PATRÓN DECORATOR (TRANSACTION)

7.3.1 Descripción del patrón

El patrón Decorator es un patrón de diseño estructural cuyo objetivo es **agregar nuevas responsabilidades a un objeto de manera dinámica**, sin modificar su clase original.

Se utiliza especialmente cuando:

- Se requiere añadir funcionalidades adicionales sin alterar el código existente.
- Se desea evitar el uso excesivo de herencia para extender comportamientos.
- Se busca mantener flexibilidad en la combinación de funcionalidades.
- Se pretende aplicar múltiples comportamientos de forma dinámica en tiempo de ejecución.

En el proyecto Enerlink, el patrón Decorator se aplicó en el módulo de energía para extender el comportamiento de las transacciones, permitiendo agregar funcionalidades adicionales sin modificar la clase original.

Esto permitió incorporar funcionalidades como validaciones, descuentos, costos adicionales y auditoría de manera flexible y desacoplada.

7.3.2 Aplicación del patrón en el proyecto Enerlink

El módulo de energía del sistema Enerlink gestiona las transacciones relacionadas con la compra y venta de energía.

Antes de aplicar el patrón Decorator, cualquier nueva funcionalidad asociada a una transacción implicaba:

- Modificar la clase Transaction
- Aumentar el acoplamiento
- Dificultar la escalabilidad del sistema
- Generar clases con múltiples responsabilidades

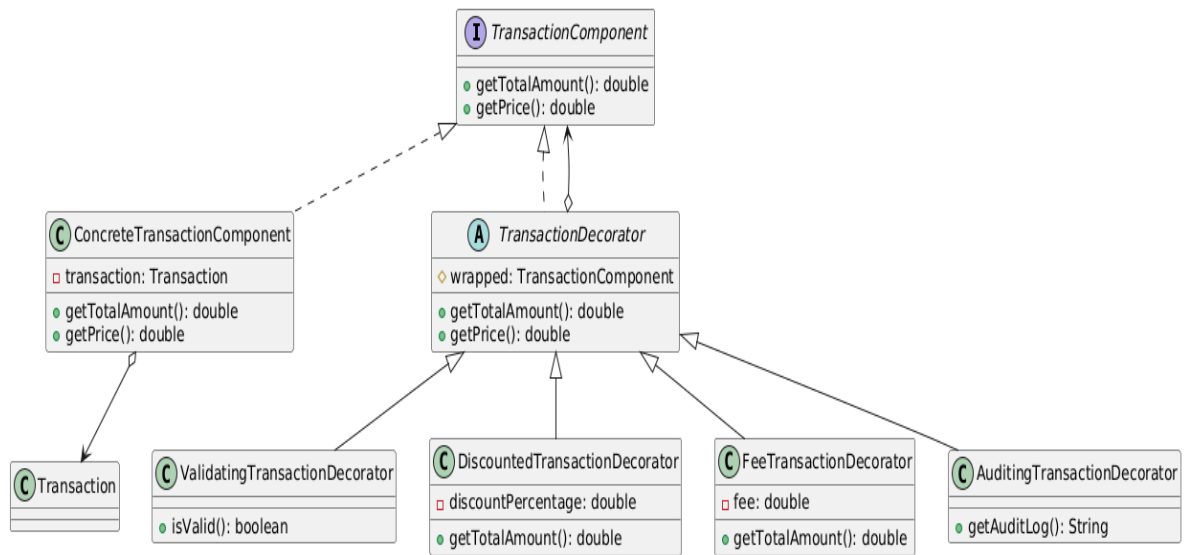
Para resolver este problema, se implementó el patrón Decorator definiendo:

1. **Componente (Component):** la interfaz TransactionComponent, que define las operaciones principales de una transacción.
2. **Componente Concreto (Concrete Component):**
 - ConcreteTransactionComponent, el cual adapta la clase existente Transaction a la interfaz del componente.
3. **Decorator Base (Decorator):**
 - TransactionDecorator, una clase abstracta que implementa TransactionComponent y contiene una referencia al componente envuelto.
4. **Decoradores Concretos (Concrete Decorators):**
 - ValidatingTransactionDecorator
 - DiscountedTransactionDecorator
 - FeeTransactionDecorator
 - AuditingTransactionDecorator

Gracias a esta estructura, es posible agregar múltiples funcionalidades a una transacción de forma dinámica, sin necesidad de modificar la clase original ni crear combinaciones específicas de clases.

7.3.3 Diagrama UML

El diagrama UML representa de manera simplificada la implementación del patrón Decorator en el módulo de energía de Enerlink:



7.3.4 Implementación del Decorator

Interfaz TransactionComponent

```

1  public interface TransactionComponent {
2
3      Long getId();
4
5      EnergyOffer getOffer();
6
7      User getBuyer();
8
9      User getSeller();
10
11     double getKwh();
12
13     double getPrice();
14
15     double getTotalAmount();
16
17     LocalDateTime getTimestamp();
18 }

```


Clase ConcreteTransactionComponent

```
1 public class ConcreteTransactionComponent implements TransactionComponent {
2
3     private final Transaction transaction;
4
5     public ConcreteTransactionComponent(Transaction transaction) {
6         this.transaction = transaction;
7     }
8
9     @Override
10    public Long getId() {
11        return transaction.getId();
12    }
13
14    @Override
15    public EnergyOffer getOffer() {
16        return transaction.getOffer();
17    }
18
19    @Override
20    public User getBuyer() {
21        return transaction.getBuyer();
22    }
23
24    @Override
25    public User getSeller() {
26        return transaction.getSeller();
27    }
28 }
```

Clase abstracta TransactionDecorator

```
1 public abstract class TransactionDecorator implements TransactionComponent {
2
3     protected final TransactionComponent wrappedComponent;
4
5     protected TransactionDecorator(TransactionComponent wrappedComponent) {
6         if (wrappedComponent == null) {
7             throw new IllegalArgumentException("Wrapped component cannot be null");
8         }
9         this.wrappedComponent = wrappedComponent;
10    }
11
12    @Override
13    public Long getId() {
14        return wrappedComponent.getId();
15    }
16
17    @Override
18    public EnergyOffer getOffer() {
19        return wrappedComponent.getOffer();
20    }
21
22    @Override
23    public User getBuyer() {
24        return wrappedComponent.getBuyer();
25    }
26 }
```

ValidatingTransactionDecorator

```
1 public class ValidatingTransactionDecorator extends TransactionDecorator {
2
3     private final List<String> validationErrors;
4
5     public ValidatingTransactionDecorator(TransactionComponent wrappedComponent) {
6         super(wrappedComponent);
7         this.validationErrors = new ArrayList<>();
8     }
9
10    @Override
11    public User getBuyer() {
12        User buyer = super.getBuyer();
13        validateBuyer(buyer);
14        return buyer;
15    }
```

DiscountedTransactionDecorator

```
1 public class DiscountedTransactionDecorator extends TransactionDecorator {
2
3     private final double discountPercentage;
4
5     public DiscountedTransactionDecorator(TransactionComponent wrappedComponent, double discountPercentage) {
6         super(wrappedComponent);
7         if (discountPercentage < 0 || discountPercentage > 100) {
8             throw new IllegalArgumentException("Discount percentage must be between 0 and 100");
9         }
10        this.discountPercentage = discountPercentage;
11    }
12
13    @Override
14    public double getPrice() {
15        double basePrice = super.getPrice();
16        double discountAmount = basePrice * (discountPercentage / 100.0);
17        return basePrice - discountAmount;
18    }
```

FeeTransactionDecorator

```
1 public class FeeTransactionDecorator extends TransactionDecorator {
2
3     private final double fixedFee;
4     private final double percentageFee;
5
6     public FeeTransactionDecorator(TransactionComponent wrappedComponent, double fixedFee) {
7         this(wrappedComponent, fixedFee, 0.0);
8     }
9
10    public FeeTransactionDecorator(TransactionComponent wrappedComponent, double fixedFee, double percentageFee) {
11        super(wrappedComponent);
12        if (fixedFee < 0) {
13            throw new IllegalArgumentException("Fixed fee cannot be negative");
14        }
15        if (percentageFee < 0 || percentageFee > 100) {
16            throw new IllegalArgumentException("Percentage fee must be between 0 and 100");
17        }
18        this.fixedFee = fixedFee;
19        this.percentageFee = percentageFee;
20    }
```

AuditingTransactionDecorator

```
1 public class AuditingTransactionDecorator extends TransactionDecorator {
2
3     private final List<String> auditLog;
4     private final DateTimeFormatter formatter;
5
6     public AuditingTransactionDecorator(TransactionComponent wrappedComponent) {
7         super(wrappedComponent);
8         this.auditLog = new ArrayList<>();
9         this.formatter = DateTimeFormatter.ISO_LOCAL_DATE_TIME;
10    }
11
12    @Override
13    public Long getId() {
14        Long id = super.getId();
15        logAudit("GET_ID", "Retrieved transaction ID: " + id);
16        return id;
17    }
18 }
```

7.3.6 Justificación de la implementación

Técnicamente:

- Permite extender el comportamiento de las transacciones sin modificar su implementación original.
- Evita la creación de múltiples subclases para cada combinación de funcionalidades.
- Favorece la reutilización de código mediante composición.
- Mejora la testabilidad al permitir probar cada comportamiento de forma independiente.
- Mantiene coherencia con la arquitectura hexagonal y los principios SOLID.

Desde la perspectiva de negocio:

- Permite agregar nuevas reglas de negocio (descuentos, validaciones, auditorías) de forma flexible.
- Facilita la evolución del sistema ante nuevos requerimientos.
- Permite adaptar el comportamiento de las transacciones según diferentes escenarios (por ejemplo, distintos tipos de usuarios o mercados).
- Mejora la mantenibilidad del sistema al separar claramente las responsabilidades.

Relación con Clean Architecture y SOLID:

La implementación respeta la arquitectura hexagonal, ubicando los decoradores dentro del dominio.

Se cumple el principio de inversión de dependencias (DIP), ya que se depende de la abstracción TransactionComponent.

Se respeta el principio de abierto/cerrado (OCP), permitiendo agregar nuevos decoradores sin modificar el código existente.

Se mantiene el principio de responsabilidad única (SRP), ya que cada decorador tiene una función específica.

7.3.7 Aclaración sobre la implementación actual

Actualmente, la implementación del patrón Decorator en el módulo de energía se centra en:

- Extender el comportamiento de las transacciones sin modificar la clase Transaction.
- Permitir la combinación dinámica de múltiples funcionalidades mediante encadenamiento de decoradores.
- Mantener la compatibilidad con otros patrones ya implementados, como el Fluent Builder y el Prototype.

El patrón se utiliza principalmente para encapsular funcionalidades transversales como validación, cálculo de precios y auditoría, manteniendo una estructura flexible y desacoplada.

Cabe resaltar que esta implementación no introduce lógica compleja innecesaria, enfocándose en una solución simple, clara y alineada con la arquitectura del sistema.

A futuro, esta implementación puede evolucionar para incluir:

- Nuevos decoradores para reglas de negocio adicionales.
- Configuración dinámica de decoradores según el contexto de ejecución.
- Integración con mecanismos de monitoreo o logging más avanzados.

7.3.8 de pruebas unitarias

Con el objetivo de validar la correcta implementación del patrón Decorator en el módulo de energía, se desarrollaron pruebas unitarias utilizando JUnit 5.

Estas pruebas permiten verificar:

- La correcta extensión dinámica del comportamiento de las transacciones.
- La delegación adecuada hacia el componente envuelto.
- El funcionamiento individual de cada decorador.
- La capacidad de combinar múltiples decoradores sin afectar la integridad del sistema.

A continuación, se presentan los casos de prueba más representativos que evidencian el correcto funcionamiento del patrón Decorator en el sistema.

7.3.8.1 Encadenamiento de múltiples decoradores

Esta prueba válida que es posible aplicar múltiples decoradores sobre una misma transacción, combinando sus comportamientos de manera dinámica.

Durante la ejecución, se encadenan los siguientes decoradores:

- ValidatingTransactionDecorator
- DiscountedTransactionDecorator
- FeeTransactionDecorator
- AuditingTransactionDecorator

El resultado esperado es que:

- El precio se vea afectado por el descuento aplicado.
- El total incluya el costo adicional (fee).
- La estructura mantenga la coherencia del cálculo final.

Esta prueba demuestra la principal ventaja del patrón Decorator, permitiendo agregar múltiples funcionalidades sin modificar la clase base.

```
1  @Test
2      @DisplayName("Should stack multiple decorators correctly")
3      void shouldStackMultipleDecoratorsCorrectly() {
4          TransactionComponent wrapped = component;
5          wrapped = new ValidatingTransactionDecorator(wrapped);
6          wrapped = new DiscountedTransactionDecorator(wrapped, 15.0);
7          wrapped = new FeeTransactionDecorator(wrapped, 25.0);
8          wrapped = new AuditingTransactionDecorator(wrapped);
9
10         double expectedPrice = 150.0 * 0.85;
11         double expectedTotal = (100.0 * expectedPrice) + 25.0;
12
13         assertEquals(expectedPrice, wrapped.getPrice(), 0.001);
14         assertEquals(expectedTotal, wrapped.getTotalAmount(), 0.001);
15     }
```

7.3.8.2 Aplicación de descuentos en la transacción

Esta prueba válida el comportamiento del DiscountedTransactionDecorator, el cual modifica el precio y el valor total de la transacción.

Durante la ejecución, se aplica un porcentaje de descuento sobre el precio base.

El resultado esperado es:

- El precio refleje correctamente el descuento aplicado.
- El total de la transacción se ajuste proporcionalmente al nuevo precio.

Esta prueba evidencia cómo un decorador puede modificar el comportamiento de un componente sin alterar su implementación original.

```

1  @Test
2      @DisplayName("Should apply 10% discount correctly")
3      void shouldApply10PercentDiscountCorrectly() {
4          TransactionComponent discounted = new DiscountedTransactionDecorator(component, 10.0);
5
6          assertEquals(135.0, discounted.getPrice(), 0.001);
7          assertEquals(13500.0, discounted.getTotalAmount(), 0.001);
8      }

```

7.3.8.3 Delegación de comportamiento al componente base

Esta prueba verifica que los decoradores delegan correctamente las operaciones que no modifican directamente.

Durante la ejecución, se utiliza un decorador de validación (ValidatingTransactionDecorator) y se invocan métodos como:

- getId
- getKwh
- getPrice
- getTotalAmount

El resultado esperado es que:

- Los valores retornados coincidan con los del componente original.
- No exista alteración en el comportamiento base cuando el decorador no lo modifica.

Esta prueba confirma el uso correcto de la composición, donde el decorador actúa como intermediario sin romper el flujo original.

```

1  @Test
2      @DisplayName("Should delegate other methods to wrapped component")
3      void shouldDelegateOtherMethodsToWrappedComponent() {
4          TransactionComponent validated = new ValidatingTransactionDecorator(component);
5
6          assertEquals(100L, validated.getId());
7          assertEquals(100.0, validated.getKwh());
8          assertEquals(150.0, validated.getPrice());
9          assertEquals(15000.0, validated.getTotalAmount());
10 }

```

7.3.8.4 Registro de auditoría en la transacción

Esta prueba valida el comportamiento del AuditingTransactionDecorator, encargado de registrar información de auditoría sobre las operaciones realizadas.

Durante la ejecución, se invocan métodos como:

- getTotalAmount
- getPrice

- getKwh

El resultado esperado es:

- Se generen registros en el log de auditoría.
- El tamaño del registro aumente conforme se ejecutan operaciones.
- Los valores originales de la transacción se mantengan intactos.

Esta prueba demuestra cómo el patrón Decorator permite agregar funcionalidades transversales (como logging) sin afectar la lógica principal.

```

1  @Test
2  @DisplayName("Should log audit entries for getTotalAmount")
3  void shouldLogAuditEntriesForGetTotalAmount() {
4      TransactionComponent audited = new AuditingTransactionDecorator(component);
5      audited.getTotalAmount();
6
7      AuditingTransactionDecorator decorator = (AuditingTransactionDecorator) audited;
8      assertTrue(decorator.getAuditLogSize() > 0);
9  }

```

7.3.8.5 Ejecución de los test unitarios

Para validar la implementación del patrón Decorator, se ejecutaron las pruebas unitarias dentro del entorno de desarrollo utilizando JUnit 5.

Los resultados obtenidos fueron satisfactorios, evidenciando que:

- Los decoradores extienden correctamente el comportamiento de las transacciones.
- La delegación hacia el componente base se realiza de forma adecuada.
- Es posible combinar múltiples decoradores sin generar conflictos.
- El sistema permite agregar nuevas funcionalidades sin modificar el código existente.

```

[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running Decorator Pattern Tests - Transaction
[INFO] Running AuditingTransactionDecorator Tests
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.083 s -- in AuditingTransactionDecorator Tests
[INFO] Running FeeTransactionDecorator Tests
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.021 s -- in FeeTransactionDecorator Tests
[INFO] Running ValidatingTransactionDecorator Tests
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.015 s -- in ValidatingTransactionDecorator Tests
[INFO] Running Edge Cases Tests
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.036 s -- in Edge Cases Tests
[INFO] Running DiscountedTransactionDecorator Tests
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.034 s -- in DiscountedTransactionDecorator Tests
[INFO] Running Decorator Stacking Tests
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.021 s -- in Decorator Stacking Tests
[INFO] Running ConcreteTransactionComponent Tests
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.010 s -- in ConcreteTransactionComponent Tests
[INFO] Tests run: 27, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.283 s -- in Decorator Pattern Tests - Transaction
[INFO] Results:
[INFO] Tests run: 27, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.907 s
[INFO] Finished at: 2026-04-09T14:42:39-05:00
[INFO] -----

```

7.4. PATRÓN FACADE (ENERGIA)

7.4.1 Descripción del patrón

El patrón **Facade** es un patrón de diseño estructural del Gang of Four cuyo objetivo es proporcionar una interfaz unificada a un conjunto de interfaces de un subsistema. Su propósito central es definir una interfaz de alto nivel que simplifique el uso del subsistema, protegiendo a los clientes de la complejidad interna de sus componentes.

Su intención formal es: *proporcionar una interfaz unificada a un conjunto de interfaces en un subsistema. Facade define una interfaz de más alto nivel que hace el subsistema más fácil de usar.*

El patrón Facade resulta apropiado en los siguientes escenarios:

- Cuando se desea proporcionar una interfaz simple y coherente para acceder a un subsistema complejo.
- Cuando existen muchas dependencias entre los clientes y las clases internas de una abstracción, aumentando el acoplamiento.
- Cuando se requiere estructurar un sistema en capas bien definidas, usando una fachada como punto de entrada a cada nivel.
- Cuando se necesita reducir la curva de aprendizaje para nuevos desarrolladores que deben interactuar con el sistema.

7.4.2 Aplicación del patrón en el proyecto Enerlink

El módulo energía de Enerlink gestiona la negociación peer-to-peer de excedentes de energía, un dominio que involucra múltiples subsistemas con responsabilidades distintas y relaciones complejas entre sí:

- Gestión de ofertas de energía: EnergyOfferService
- Selección y ejecución de procesos de venta: DirectSaleProcess, AuctionSaleProcess
- Creación de procesos mediante fábricas: DirectSaleFactory, AuctionSaleFactory
- Cadena de decoradores sobre transacciones: validación, tarifas y auditoría
- Persistencia de ofertas: EnergyOfferRepositoryPort

Antes de aplicar el patrón Facade, el controlador EnergyOfferController debía coordinar directamente todos estos subsistemas. Esto implicaba que el controlador conociera la cadena de decoradores, seleccionara la fábrica correcta según el tipo de oferta y administrara la lógica de ejecución del proceso de venta. Esta situación generaba:

- Alto acoplamiento entre la capa de infraestructura y el dominio del negocio.
- Violación del principio de responsabilidad única en el controlador REST.
- Dificultad para escribir pruebas unitarias aisladas.
- Fragilidad ante cambios en los subsistemas internos.

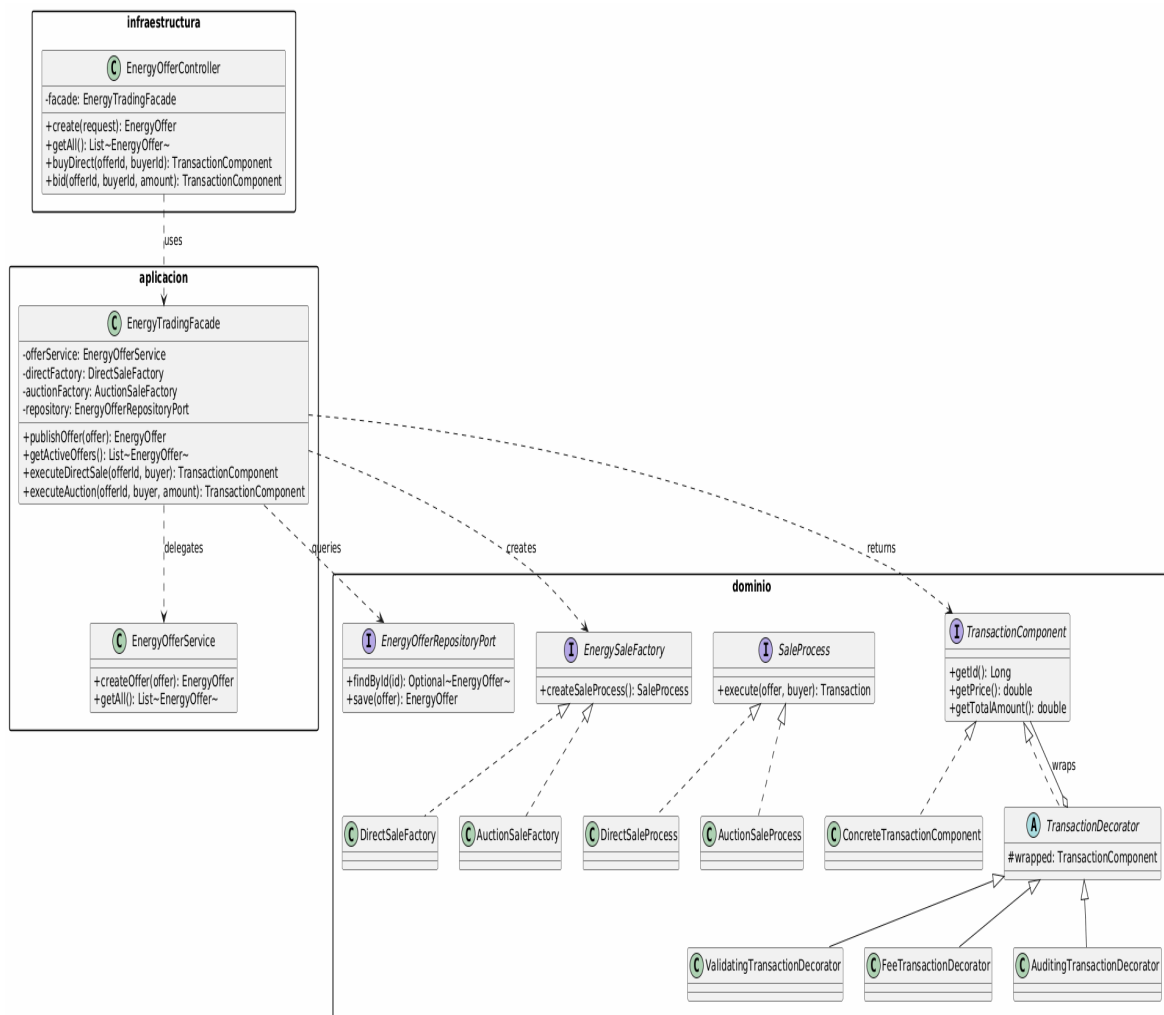
Para resolver este problema, se implementó el patrón Facade definiendo la clase EnergyTradingFacade como único punto de entrada al módulo de energía, la cual oculta:

- La lógica de selección del proceso de venta según el tipo de oferta.
- La aplicación de la cadena de decoradores a cada transacción.
- La coordinación entre el servicio de ofertas, las fábricas y el repositorio.
- Los detalles de construcción y enriquecimiento de los objetos de transacción.

Como resultado, el controlador pasó de depender de múltiples clases a depender de una sola: EnergyTradingFacade. Toda la complejidad de coordinación quedó encapsulada en la capa de aplicación, respetando el aislamiento hexagonal.

7.4.3 Diagrama UML

El diagrama UML representa de manera simplificada la implementación del patrón Facade en el módulo **energía** de Enerlink, mostrando la relación entre el controlador cliente, la fachada central y los subsistemas del dominio que coordina:



7.4.4 Implementación del Facade

Cliente EnergyOfferController

```
1 public class EnergyOfferController {
2
3     private final EnergyTradingFacade facade;
4
5     public EnergyOfferController(EnergyTradingFacade facade) {
6         this.facade = facade;
7     }
8
9     @PostMapping
10    public ResponseEntity<EnergyOffer> create(
11        @RequestBody EnergyOfferRequest request) {
12
13        EnergyOffer offer = facade.publishOffer(
14            request.getSaleType(),
15            request.getProducerId(),
16            request.getKwh(),
17            request.getPrice());
18
19        return ResponseEntity.ok(offer);
20    }
21
22    @GetMapping
23    public ResponseEntity<List<EnergyOffer>> getAll() {
24        return ResponseEntity.ok(facade.getActiveOffers());
25    }
26
27    @GetMapping("/{id}")
28    public ResponseEntity<EnergyOffer> getById(@PathVariable Long id) {
29        EnergyOffer offer = facade.getOfferById(id);
30        return ResponseEntity.ok(offer);
31    }
32 }
```

Clase EnergyTradingFacade

```
1 public class EnergyTradingFacade {
2
3     private final EnergyOfferService energyOfferService;
4     private final DirectSaleFactory directSaleFactory;
5     private final AuctionSaleFactory auctionSaleFactory;
6     private final EnergyOfferRepositoryPort repositoryPort;
7
8     public EnergyTradingFacade(
9         EnergyOfferService energyOfferService,
10        DirectSaleFactory directSaleFactory,
11        AuctionSaleFactory auctionSaleFactory,
12        EnergyOfferRepositoryPort repositoryPort) {
13        this.energyOfferService = energyOfferService;
14        this.directSaleFactory = directSaleFactory;
15        this.auctionSaleFactory = auctionSaleFactory;
16        this.repositoryPort = repositoryPort;
17    }
18
19    public EnergyOffer publishOffer(SaleType saleType, Long producerId, double kwh, double price) {
20        return energyOfferService.createOffer(saleType, producerId, kwh, price);
21    }
22
23    public Transaction executeDirectSale(Long offerId, User buyer) {
24        EnergyOffer offer = repositoryPort.findById(offerId)
25            .orElseThrow(() -> new RuntimeException("Oferta no encontrada con id: " + offerId));
26
27        if (offer.getSaleType() != SaleType.DIRECT) {
28            throw new IllegalStateException("La oferta no es de tipo venta directa. Tipo actual: " + offer.getSaleType());
29        }
30
31        SaleProcess saleProcess = directSaleFactory.createSaleProcess();
32        Transaction transaction = saleProcess.execute(offer, buyer, offer.getKwh());
33
34        return applyDecoratorChain(transaction);
35    }
36 }
```

7.4.5 Justificación de la implementación

Técnicamente

- Permite reducir el acoplamiento entre la capa de infraestructura y los subsistemas del dominio, ya que el controlador solo conoce la interfaz pública del Facade.
- Centraliza la aplicación de la cadena de decoradores, garantizando que todas las transacciones pasen por el mismo proceso de enriquecimiento independientemente de su origen.
- Simplifica las pruebas unitarias del controlador, que ahora solo necesita mockear una única clase en lugar de múltiples dependencias.
- Facilita la evolución del sistema: modificar un subsistema interno no requiere cambios en el controlador.
- Mantiene coherencia con la arquitectura hexagonal al ubicar el Facade en la capa de aplicación, sin importar dependencias hacia la infraestructura.

Desde la perspectiva de negocio

- Unifica el punto de acceso a las operaciones de comercio de energía, reduciendo la posibilidad de omitir pasos críticos como la validación o la auditoría.
- Permite introducir nuevas reglas de negocio (nuevos decoradores, nuevos tipos de venta) sin modificar el controlador ni el contrato de la API REST.
- Mejora la legibilidad del código para desarrolladores que se incorporen al equipo, quienes pueden entender el flujo completo leyendo únicamente los métodos del Facade.
- Facilita la auditoría y el cumplimiento normativo al garantizar que toda transacción pase siempre por el `AuditingTransactionDecorator`.

Relación con Clean Architecture y SOLID

La implementación respeta la arquitectura hexagonal ubicando el Facade en la capa aplicación/, dependiendo únicamente de puertos del dominio y nunca de detalles de infraestructura.

Se cumple el principio de responsabilidad única (SRP): el Facade tiene una única razón para cambiar, que es la coordinación entre subsistemas, separada de la lógica de cada uno de ellos.

Se respeta el principio abierto/cerrado (OCP): agregar una nueva operación al Facade no requiere modificar las existentes, y agregar un nuevo subsistema tampoco afecta al controlador.

Se aplica el principio de inversión de dependencias (DIP): el Facade depende de las abstracciones `EnergyOfferRepositoryPort` y `EnergySaleFactory`, no de sus implementaciones concretas.

7.4.6 Aclaración sobre la implementación actual

Actualmente, la implementación del patrón Facade en el módulo de energía se centra en:

- Proporcionar un único punto de entrada al sistema de comercio energético, simplificando la interacción del controlador con el dominio.
- Encapsular la selección del proceso de venta, el uso de las fábricas y la construcción de la cadena de decoradores, ocultando esa complejidad de la capa de infraestructura.
- Mantener compatibilidad total con los patrones ya implementados (Abstract Factory, Fluent Builder, Decorator, Prototype) sin modificar ninguna de sus clases.
- Respetar el aislamiento de la arquitectura hexagonal: el Facade vive en aplicación/, nunca importa clases de infraestructura/.

Cabe resaltar que esta implementación no introduce lógica de negocio propia en el Facade. Su rol es exclusivamente de coordinación y simplificación, evitando convertirse en un God Object.

A futuro, esta implementación puede evolucionar para incluir:

- Nuevos métodos que coordinen operaciones combinadas, como la compra por lote de múltiples ofertas.
- Integración con un módulo de notificaciones, donde el Facade también dispare eventos tras una transacción exitosa.
- Soporte para transacciones distribuidas si el sistema evoluciona hacia una arquitectura de microservicios.
- Configuración dinámica de la cadena de decoradores según el perfil del usuario o el tipo de mercado energético.

En cualquiera de estos escenarios, la estructura del Facade permite incorporar estas mejoras añadiendo métodos o ajustando la coordinación interna, sin modificar el controlador ni los subsistemas del dominio.

7.4.7 Pruebas unitarias

Con el objetivo de validar la correcta implementación del patrón Facade en el módulo de energía, se desarrollaron pruebas unitarias utilizando JUnit 5.

Estas pruebas permiten verificar:

- La correcta delegación de operaciones simples hacia los subsistemas correspondientes.
- La coordinación adecuada entre múltiples subsistemas durante una operación compleja.
- La aplicación completa de la cadena de decoradores sobre la transacción resultante.
- El desacoplamiento efectivo del controlador respecto a los subsistemas internos del dominio.

A continuación, se presentan los casos de prueba más representativos que evidencian el correcto funcionamiento del patrón Facade en el sistema.

7.4.7.1 Delegación de operaciones simples al subsistema correspondiente

Esta prueba valida que el Facade delega correctamente las operaciones simples hacia el subsistema adecuado, sin introducir lógica adicional.

Durante la ejecución, se invoca el método `publishOffer()` del Facade con los parámetros correspondientes a una nueva oferta de energía.

El resultado esperado es que:

- El método `createOffer()` de `EnergyOfferService` sea invocado exactamente una vez.
- Los argumentos recibidos por el servicio coincidan con los proporcionados originalmente al Facade.
- El Facade retorne el mismo objeto que devuelve el servicio, sin modificaciones.

Esta prueba confirma que el Facade cumple su rol de intermediario en operaciones de delegación directa, sin añadir comportamiento propio ni alterar los datos transmitidos.

```
1  @Test
2      @DisplayName("Should delegate to EnergyOfferService with correct arguments")
3      void should_delegate_to_energy_offer_service_with_correct_arguments() {
4          when(energyOfferService.createOffer(SaleType.DIRECT, 1L, 100.0, 150.0))
5              .thenReturn(directOffer);
6
7          facade.publishOffer(SaleType.DIRECT, 1L, 100.0, 150.0);
8
9          verify(energyOfferService).createOffer(SaleType.DIRECT, 1L, 100.0, 150.0);
10     }
```

7.4.7.2 Coordinación de múltiples subsistemas en una venta directa

Esta prueba valida el comportamiento de orquestación del Facade al ejecutar una venta directa, donde deben coordinarse en secuencia el repositorio, la fábrica y el proceso de venta.

Durante la ejecución, se invoca `executeDirectSale()` con un identificador de oferta válido y un comprador. Internamente el Facade debe:

- Consultar el repositorio para obtener la oferta.
- Solicitar a `DirectSaleFactory` la creación del proceso de venta.
- Invocar el proceso con los parámetros correctos: la oferta, el comprador y la cantidad en kWh.

El resultado esperado es que:

- El proceso de venta reciba exactamente los parámetros (directOffer, buyer, 100.0).
- Cada subsistema sea invocado una única vez en el orden correcto.
- El controlador no necesite conocer ni coordinar ninguno de estos pasos por su cuenta.

Esta prueba demuestra la principal ventaja del patrón Facade: ocultar la complejidad de la coordinación entre subsistemas detrás de una interfaz de invocación simple.

```
1  @Test
2  @DisplayName("Should execute the sale process with correct parameters")
3  void should_execute_sale_process_with_correct_parameters() {
4      when(repositoryPort.findById(1L)).thenReturn(Optional.of(directOffer));
5      when(directSaleFactory.createSaleProcess()).thenReturn(saleProcess);
6
7      facade.executeDirectSale(1L, buyer);
8
9      verify(saleProcess).execute(directOffer, buyer, 100.0);
10 }
```

7.4.7.3 Aplicación completa de la cadena de decoradores sobre la transacción

Esta prueba valida que el Facade aplica correctamente la cadena de decoradores sobre la transacción generada por el proceso de venta, garantizando que el resultado final sea una transacción enriquecida e íntegra.

Durante la ejecución, se invoca executeDirectSale() y se verifica la transacción retornada tras el encadenamiento de:

- ValidatingTransactionDecorator
- FeeTransactionDecorator
- AuditingTransactionDecorator

El resultado esperado es que:

- La transacción retornada no sea nula.
- Los campos id y timestamp estén presentes, evidenciando que el objeto fue construido y enriquecido correctamente.
- Los valores de offer y buyer correspondan a los datos originales de la operación.

Esta prueba evidencia que el Facade encapsula por completo la responsabilidad de construir y aplicar la cadena de decoradores, garantizando que ninguna transacción escape a este proceso independientemente de su origen.

```

1  @Test
2      @DisplayName("Should wrap transaction with all decorators in correct order")
3      void should_wrap_transaction_with_all_decorators() {
4          when(repositoryPort.findById(1L)).thenReturn(Optional.of(directOffer));
5          when(directSaleFactory.createSaleProcess()).thenReturn(saleProcess);
6
7          Transaction result = facade.executeDirectSale(1L, buyer);
8
9          assertNotNull(result);
10         assertNotNull(result.getId());
11         assertNotNull(result.getTimestamp());
12         assertEquals(directOffer, result.getOffer());
13         assertEquals(buyer, result.getBuyer());
14     }

```

7.4.7.4 Desacoplamiento del controlador respecto a los subsistemas internos

Esta prueba valida que el controlador queda completamente desacoplado de los subsistemas del dominio, dependiendo únicamente del Facade para acceder a todas las operaciones de comercio energético.

Durante la ejecución, se instancia `EnergyTradingFacade` inyectando sus cuatro dependencias por constructor y se verifica que la instancia resultante sea válida y funcional.

El resultado esperado es que:

- La instancia del Facade sea creada exitosamente con una única inyección de dependencias.
- Todas las operaciones de trading sean accesibles a través del Facade sin necesidad de referenciar ningún subsistema directamente.

Esta prueba confirma el beneficio estructural del patrón: el controlador pasa de gestionar múltiples dependencias a depender de un único punto de entrada, reduciendo el acoplamiento y simplificando tanto el código de producción como las pruebas del controlador.

```

1  @Test
2      @DisplayName("Facade should be the only dependency for trading operations")
3      void facade_should_be_only_dependency() {
4          EnergyTradingFacade facadeInstance = new EnergyTradingFacade(
5              energyOfferService, directSaleFactory, auctionSaleFactory, repositoryPort);
6
7          assertNotNull(facadeInstance);
8      }

```

7.4.7.5 Ejecución de los test unitarios

Para validar la implementación del patrón Facade, se ejecutaron las pruebas unitarias dentro del entorno de desarrollo utilizando JUnit 5.

Los resultados obtenidos fueron satisfactorios, evidenciando que:

- El Facade delega correctamente las operaciones simples hacia los subsistemas correspondientes.
- La coordinación entre repositorio, fábrica y proceso de venta se realiza en el orden correcto y con los parámetros esperados.
- La cadena de decoradores es aplicada de forma íntegra sobre cada transacción generada.
- El controlador queda completamente desacoplado de los subsistemas internos del dominio, dependiendo únicamente del Facade.

```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running EnergyTradingFacade Tests
[INFO] Running executeDirectSale Tests
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockito.html#0.3
WARNING: A Java agent has been loaded dynamically (C:\Users\estadistica\.m2\repository\net\bytebuddy\byte-buddy-agent\1.17.8\byte-buddy-agent-1.17.8.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
Venta directa completada: 15000.0
Venta directa completada: 15000.0
Venta directa completada: 15000.0
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.470 s -- in executeDirectSale Tests
[INFO] Running Edge Case Tests
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.044 s -- in Edge Case Tests
[INFO] Running Controller Decoupling Tests
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.024 s -- in Controller Decoupling Tests
[INFO] Running publishOffer Tests
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.020 s -- in publishOffer Tests
[INFO] Running executeAuction Tests
Venta directa completada con precio personalizado: 15000.0
Venta directa completada con precio personalizado: 15000.0
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.029 s -- in executeAuction Tests
[INFO] Running Decorator Chain Integration Tests
Venta directa completada: 15000.0
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.007 s -- in Decorator Chain Integration Tests
[INFO] Running getActiveOffers Tests
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.011 s -- in getActiveOffers Tests
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.647 s -- in EnergyTradingFacade Tests
[INFO] Results:
[INFO]
[INFO] Tests run: 19, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 5.433 s
[INFO] Finished at: 2026-04-15T09:59:48-05:00
[INFO] -----
```


7.5. PATRÓN COMPOSITE (IOT)

7.5.1 Descripción del patrón

El patrón **Composite** es un patrón de diseño estructural del Gang of Four cuyo objetivo es componer objetos en estructuras de árbol para representar jerarquías parte-todo. Su propósito central es que los clientes puedan tratar objetos individuales y composiciones de objetos de manera completamente uniforme, sin distinguir si están interactuando con un elemento simple o con un grupo de elementos.

Su intención formal es: *componer objetos en estructuras de árbol para representar jerarquías parte-todo. Composite permite a los clientes tratar objetos individuales y composiciones de objetos de manera uniforme.*

El patrón Composite resulta apropiado en los siguientes escenarios:

- Cuando se necesita representar jerarquías de objetos donde los clientes deben tratar elementos simples y compuestos de la misma manera.
- Cuando el código cliente no debe depender de si está trabajando con uno o múltiples objetos del mismo tipo.
- Cuando se requiere agregar operaciones sobre un conjunto de objetos de forma transparente, sin exponer la estructura interna al llamador.
- Cuando se busca facilitar la extensión del sistema añadiendo nuevas hojas sin modificar el código existente.

7.5.2 Aplicación del patrón en el proyecto Enerlink

El módulo **iot** de Enerlink gestiona la obtención de datos de dispositivos IoT a través de múltiples APIs de marcas en la nube. El sistema no se comunica directamente con los dispositivos físicos; en su lugar, consulta la API de cada fabricante a través de un adaptador dedicado:

- **SmartHome API:** datos de dispositivos de la marca SmartHome, consumidos a través de SmartHomeAdapter
- **EnergyCloud API:** datos de dispositivos de la marca EnergyCloud, consumidos a través de EnergyCloudAdapter

Antes de aplicar el patrón Composite, el servicio IoTProcessorService debía conocer y gestionar cada adaptador de forma individual. Esto implicaba que el servicio tuviera dependencias directas a cada implementación concreta y que, ante la incorporación de una nueva marca de dispositivos, fuera necesario modificar el servicio. Esta situación generaba:

- Alto acoplamiento entre el servicio de procesamiento y las implementaciones concretas de cada adaptador.
- Violación del principio abierto/cerrado: agregar una nueva fuente de datos requería modificar código existente.

- Dificultad para escribir pruebas unitarias del servicio sin instanciar todos los adaptadores.
- Fragilidad estructural ante el crecimiento del número de marcas de dispositivos soportadas.

Para resolver este problema, se implementó el patrón Composite formalizando la clase `IoTDataPortComposite` como punto de entrada único al conjunto de fuentes de datos IoT. Esta clase oculta:

- El número de fuentes de datos registradas en el sistema.
- Los tipos concretos de adaptadores que proveen la información.
- La lógica de iteración y agregación de resultados provenientes de múltiples APIs.
- La forma en que se incorporan nuevas marcas al sistema.

Como resultado, `IoTProcessorService` pasó de depender de múltiples adaptadores concretos a depender de una sola abstracción.

7.5.3 Diagrama UML

El diagrama UML representa la implementación del patrón Composite en el módulo `iot` de Enerlink, mostrando los roles de cada clase Component, Leaf y Composite y sus relaciones dentro de la arquitectura hexagonal:



7.5.4 Implementación del Composite

Clase IoTDataPortComposite

```
1 public class IoTDataPortComposite implements IoTDataPort {
2
3     private static final Logger logger = LoggerFactory.getLogger(IoTDataPortComposite.class);
4
5     private final List<IoTDataPort> children;
6
7     public IoTDataPortComposite() {
8         this.children = new ArrayList<>();
9     }
10
11     public void add(IoTDataPort child) {
12         if (child != null && !children.contains(child)) {
13             children.add(child);
14             logger.debug("Adapter added to composite: {}", child.getClass().getSimpleName());
15         }
16     }
17
18     public void remove(IoTDataPort child) {
19         if (child != null) {
20             children.remove(child);
21             logger.debug("Adapter removed from composite: {}", child.getClass().getSimpleName());
22         }
23     }
24
25     public List<IoTDataPort> getChildren() {
26         return List.copyOf(children);
27     }
28 }
```

Clase IoTProcessorService

```
1 public class IoTProcessorService {
2
3     private static final Logger logger = LoggerFactory.getLogger(IoTProcessorService.class);
4
5     private final IoTProcessor defaultProcessor;
6     private final IoTDataPort ioTDataPort;
7
8     public IoTProcessorService(IoTProcessor defaultProcessor, IoTDataPort ioTDataPort) {
9         this.defaultProcessor = defaultProcessor;
10        this.ioTDataPort = ioTDataPort;
11    }
12
13    public IoTProcessor getDefaultProcessor() {
14        return defaultProcessor;
15    }
16
17    public IoTProcessor getProcessorForProvider(String provider) {
18        logger.debug("Getting processor for provider: {}", provider);
19        if (provider == null) {
20            return defaultProcessor;
21        }
22        return defaultProcessor;
23    }
24
25    public IoTDeviceData processDeviceData(String deviceId) {
26        logger.info("Processing device data for: {}", deviceId);
27        return defaultProcessor.processDeviceData(deviceId);
28    }
29 }
```

7.5.5 Justificación de la implementación

Técnicamente

- Elimina el acoplamiento entre el servicio de procesamiento y los adaptadores concretos, ya que el servicio solo conoce la interfaz IoTDataPort.
- Centraliza la lógica de agregación de datos en el Composite, evitando que esa responsabilidad recaiga en el servicio o en el controlador.
- Simplifica las pruebas unitarias del servicio, que ahora solo necesita un mock de IoTDataPort en lugar de mocks de cada adaptador individual.
- Cumple con el principio abierto/cerrado: agregar una nueva fuente de datos se reduce a crear una nueva hoja y registrarla en la configuración, sin tocar ninguna clase existente.
- Mantiene coherencia con la arquitectura hexagonal: el Composite vive en infraestructura, el Component en dominio, y el servicio en aplicación.

Desde la perspectiva de negocio

- Permite incorporar nuevas marcas de dispositivos IoT sin interrumpir el servicio ni modificar la lógica de procesamiento existente.
- Garantiza que todos los datos de todas las fuentes registradas sean consultados en cada petición, sin riesgo de omitir alguna fuente por error de implementación.
- Facilita la resiliencia del sistema: si un adaptador falla, el Composite puede gestionar el error de forma localizada y seguir retornando datos de los demás.
- Reduce el tiempo de incorporación de nuevas integraciones para el equipo de desarrollo, al proporcionar un contrato claro y una estructura de extensión predecible.

Relación con Clean Architecture y SOLID

La implementación respeta la arquitectura hexagonal ubicando el Composite en la capa infraestructura/, implementando el puerto IoTDataPort definido en el dominio, y siendo consumido por IoTProcessorService a través de esa misma abstracción.

Se cumple el **principio de responsabilidad única (SRP)**: cada adaptador solo sabe cómo obtener datos de su API, el Composite solo sabe cómo agregar resultados, y el servicio solo sabe cómo procesar los datos recibidos.

Se respeta el **principio abierto/cerrado (OCP)**: el sistema puede extenderse con nuevas fuentes de datos creando nuevas hojas, sin modificar el Composite, el servicio ni ninguna otra clase existente.

Se aplica el **principio de inversión de dependencias (DIP)**: IoTProcessorService depende de la abstracción IoTDataPort, nunca de implementaciones concretas como SmartHomeAdapter o EnergyCloudAdapter.

Relación con los patrones Adapter y Bridge del módulo

La implementación del Composite no actúa de forma aislada en el módulo **iot**; complementa y potencia los patrones ya existentes:

El patrón **Adapter** —implementado en `SmartHomeAdapter` y `EnergyCloudAdapter`— convierte la interfaz específica de cada API de marca al formato interno `IoTDataPort`. Estos mismos adaptadores son las hojas del Composite. Sin el Adapter, las hojas no existirían; sin el Composite, las hojas tendrían que ser gestionadas individualmente por el servicio.

El patrón **Bridge** —implementado en la jerarquía `IoTProcessor / AbstractIoTProcessor`— separa la abstracción del procesamiento de sus implementaciones concretas (`SimpleIoTProcessor`, `FilteringIoTProcessor`, `EnrichedIoTProcessor`). El Composite actúa en la capa de obtención de datos, justo antes de que el Bridge entre en acción para procesarlos. Los dos patrones resuelven problemas distintos dentro del mismo módulo sin interferir entre sí.

7.5.6 Aclaración sobre la implementación actual

Actualmente, la implementación del patrón Composite en el módulo IoT se centra en:

- Proporcionar un único punto de entrada al conjunto de fuentes de datos IoT, simplificando la interacción del servicio con múltiples APIs de marcas.
- Encapsular la iteración sobre los adaptadores registrados y la agregación de sus resultados, ocultando esa complejidad del servicio de aplicación.
- Mantener compatibilidad total con los patrones ya implementados en el módulo (Adapter y Bridge) sin modificar ninguna de sus clases.
- Respetar el aislamiento de la arquitectura hexagonal: el Composite vive en infraestructura/, implementa un puerto del dominio y nunca es referenciado directamente por el servicio.

Cabe resaltar que el Composite no introduce lógica de procesamiento propia. Su único rol es agregar transparentemente los datos de sus hijos y retornarlos como si provinieran de una sola fuente.

A futuro, esta implementación puede evolucionar para incluir:

- Manejo de errores por adaptador, donde si una fuente falla el Composite retorna los datos de las fuentes disponibles en lugar de lanzar una excepción global.
- Pesos o prioridades por adaptador, permitiendo configurar qué fuente tiene precedencia en caso de datos duplicados sobre el mismo dispositivo.
- Registro dinámico de adaptadores en tiempo de ejecución, sin necesidad de reiniciar la aplicación al incorporar una nueva marca.
- Soporte para estructuras Composite anidadas, donde un Composite puede contener a su vez otros Composites que agrupen marcas por región o por tipo de dispositivo.

7.5.7 Pruebas unitarias

Con el objetivo de validar la correcta implementación del patrón Composite en el módulo IoT, se desarrollaron pruebas unitarias utilizando JUnit 5.

Estas pruebas permiten verificar:

- La correcta agregación de resultados provenientes de múltiples adaptadores hijos.
- El tratamiento uniforme del Composite a través de la interfaz Component, sin distinción del cliente.
- La gestión dinámica de adaptadores y su efecto sobre los resultados retornados.

A continuación, se presentan los casos de prueba más representativos que evidencian el correcto funcionamiento del patrón Composite en el sistema.

7.5.7.1 Agregación de resultados de múltiples adaptadores

Esta prueba valida el comportamiento central del patrón Composite: delegar la operación a todos los hijos registrados y consolidar sus resultados en una única respuesta.

Durante la ejecución, se registran tres adaptadores en el Composite y se invoca el método `fetchAllDevices()`. Cada adaptador retorna un dispositivo distinto.

El resultado esperado es que:

- Los tres adaptadores sean invocados exactamente una vez cada uno.
- La lista retornada contenga los dispositivos de todas las fuentes agregados.

Esta prueba demuestra la principal ventaja del patrón Composite: el cliente interactúa con una interfaz simple y recibe resultados completos, independientemente del número de fuentes subyacentes.

```
1  @Test
2  void should_delegate_to_all_children_and_aggregate_results_when_fetchAllDevices() {
3      when(adapter1.fetchAllDevices()).thenReturn(List.of(
4          createDevice("DEV-001", "Device 1", DeviceType.SMART_METER)));
5      when(adapter2.fetchAllDevices()).thenReturn(List.of(
6          createDevice("DEV-002", "Device 2", DeviceType.THERMOSTAT)));
7      when(adapter3.fetchAllDevices()).thenReturn(List.of(
8          createDevice("DEV-003", "Device 3", DeviceType.SENSOR)));
9
10     composite.add(adapter1);
11     composite.add(adapter2);
12     composite.add(adapter3);
13
14     List<IoTDeviceData> results = composite.fetchAllDevices();
15
16     assertEquals(3, results.size());
17     verify(adapter1).fetchAllDevices();
18     verify(adapter2).fetchAllDevices();
19     verify(adapter3).fetchAllDevices();
20 }
```

7.5.7.2 Tratamiento uniforme a través de la interfaz Component

Esta prueba valida que el Composite es tratado por el cliente exactamente igual que cualquier hoja, confirmando el principio de uniformidad que define al patrón.

Durante la ejecución, se asigna el Composite a una variable de tipo `IoTDataPort` —la interfaz `Component`— y se invoca `fetchAllDevices()` a través de ella.

El resultado esperado es que:

- La operación se ejecute correctamente sin necesidad de hacer cast ni de conocer el tipo concreto.
- El adaptador hijo registrado sea invocado, demostrando que la delegación funciona a través de la abstracción.

Esta prueba confirma que el servicio cliente puede depender únicamente de `IoTDataPort` para operar, sin importar si detrás hay un adaptador simple o un Composite que agrupa múltiples fuentes.

```
1  @Test
2  void should_interact_with_port_interface_when_processing() {
3      composite.add(adapter1);
4      when(adapter1.fetchAllDevices()).thenReturn(List.of());
5
6      IoTDataPort port = composite;
7      port.fetchAllDevices();
8
9      verify(adapter1).fetchAllDevices();
10 }
```

7.5.7.3 Gestión dinámica de adaptadores y su efecto en los resultados

Esta prueba valida que el Composite refleja inmediatamente los cambios en su lista interna de hijos: al eliminar un adaptador, sus datos dejan de aparecer en las respuestas y el adaptador no vuelve a ser invocado.

Durante la ejecución, se registran dos adaptadores, se elimina uno mediante `remove()` y se invoca `fetchAllDevices()`.

El resultado esperado es que:

- Solo el adaptador que permanece en la lista sea invocado.
- La lista retornada contenga únicamente los dispositivos del adaptador restante.
- El adaptador eliminado no sea invocado en ningún momento tras su remoción.

Esta prueba evidencia que la estructura parte-todo del Composite es dinámica y coherente: la gestión de hijos se refleja directamente en el comportamiento del sistema sin necesidad de modificar ninguna otra clase.

```
1  @Test
2      void should_return_data_from_remaining_adapters_when_one_removed() {
3          when(adapter1.fetchAllDevices()).thenReturn(List.of(
4              createDevice("DEV-001", "Device 1", DeviceType.SMART_METER)));
5          when(adapter2.fetchAllDevices()).thenReturn(List.of(
6              createDevice("DEV-002", "Device 2", DeviceType.THERMOSTAT)));
7
8          composite.add(adapter1);
9          composite.add(adapter2);
10         composite.remove(adapter1);
11
12         List<IoTDeviceData> results = composite.fetchAllDevices();
13
14         assertEquals(1, results.size());
15         assertEquals("DEV-002", results.get(0).getDeviceId());
16         verify(adapter1, never()).fetchAllDevices();
17         verify(adapter2).fetchAllDevices();
18     }
```

7.5.7.4 Resiliencia ante respuestas nulas de un adaptador

Esta prueba valida que el Composite maneja correctamente el caso en que uno de sus hijos retorna null, continuando con los demás adaptadores y retornando los datos disponibles sin lanzar una excepción.

Durante la ejecución, se registran dos adaptadores: el primero retorna null al ser invocado, mientras que el segundo retorna un dispositivo válido.

El resultado esperado es que:

- No se lance ninguna excepción durante la agregación.
- La lista retornada contenga únicamente los datos del adaptador que respondió correctamente.
- El Composite absorba el fallo parcial de forma localizada, sin afectar al resto de la operación.

Esta prueba demuestra cómo el patrón Composite puede incorporar lógica de resiliencia de forma centralizada, protegiendo al servicio cliente de fallos individuales en alguna de las fuentes de datos registradas.


```

1  @Test
2  void should_handle_null_return_from_adapter_when_fetchAllDevices() {
3      when(adapter1.fetchAllDevices()).thenReturn(null);
4      when(adapter2.fetchAllDevices()).thenReturn(List.of(
5          createDevice("DEV-002", "Device 2", DeviceType.THERMOSTAT)));
6
7      composite.add(adapter1);
8      composite.add(adapter2);
9
10     List<IoTDeviceData> results = composite.fetchAllDevices();
11
12     assertEquals(1, results.size());
13     assertEquals("DEV-002", results.get(0).getDeviceId());
14 }

```

7.5.7.5 Ejecución de los test unitarios

Para validar la implementación del patrón Composite, se ejecutaron las pruebas unitarias dentro del entorno de desarrollo utilizando JUnit 5.

Los resultados obtenidos fueron satisfactorios, evidenciando que:

- El Composite delega correctamente la operación a todos los adaptadores registrados y agrega sus resultados en una respuesta unificada.
- El cliente puede operar exclusivamente a través de la interfaz IoTDataPort, sin distinción entre hojas y Composite.
- La gestión dinámica de adaptadores se refleja inmediatamente en el comportamiento del sistema.
- El Composite absorbe fallos parciales de forma localizada, manteniendo la estabilidad del sistema ante respuestas nulas de alguna fuente.

```

[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.enerlink.enerlink.iot.infraestructura.adapter.IoTDataPortCompositeTest
Mockito is currently self-attaching to enable the inline-mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build as described in Mockito's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org.mockito/org/mockito/Mockito.html#0.3
WARNING: A Java agent has been loaded dynamically (C:\Users\estadistica\m2\repository\net\bytebuddy\byte-buddy-agent\1.17.8\byte-buddy-agent-1.17.8.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 19, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.293 s -- in com.enerlink.enerlink.iot.infraestructura.adapter.IoTDataPortCompositeTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 19, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.194 s
[INFO] Finished at: 2026-04-15T15:40:45-05:00
[INFO] -----

```

10. REFERENCIAS BIBLIOGRÁFICAS

Refactoring.Guru. (s. f.). *Singleton*. <https://refactoring.guru/es/design-patterns/singleton>

Refactoring.Guru. (s. f.). *Factory Method*.

<https://refactoring.guru/es/design-patterns/factory-method>

SourceMaking. (s. f.). *Factory Method design pattern*.

https://sourcemaking.com/design_patterns/factory_method

Refactoring.Guru. (s. f.). *Abstract factory*. Refactoring.Guru. <https://refactoring.guru/design-patterns/abstract-factory>

Refactoring.Guru. (s. f.). *Builder*. <https://refactoring.guru/design-patterns/builder>

Refactoring.Guru. (s. f.). *Prototype*. <https://refactoring.guru/es/design-patterns/prototype>

Refactoring.Guru. (s. f.). *Creational patterns*. Refactoring.Guru. <https://refactoring.guru/design-patterns/creational-patterns>

Refactoring.Guru. (s. f.). *Adapter*. <https://refactoring.guru/es/design-patterns/adapter>

Refactoring.Guru. (s. f.). *Bridge*. <https://refactoring.guru/es/design-patterns/bridge>

Refactoring.Guru. (s. f.). *Decorator*. <https://refactoring.guru/es/design-patterns/decorator>

Refactoring.Guru. (s. f.). *Facade* <https://refactoring.guru/design-patterns/facade>

Refactoring.Guru. (s. f.). *Composite* <https://refactoring.guru/design-patterns/composite>

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

http://www.cse.hcmut.edu.vn/~thai/books/Design_Patterns_Elements_of_Reusable_Object-Oriented_Software.pdf

Freeman, E., & Robson, E. (2004). *Head first design patterns*. O'Reilly Media.

https://www.staff.ces.funai.edu.ng/textbook-solutions/Resources/HomePages/Head_First_Design_Patterns_Eric_Freeman.pdf

Martin, R. C. (2018). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Martin, R. C. (2003). *Agile software development: Principles, patterns, and practices*. Prentice Hall.